

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук
(повна назва)

Кафедра _____ програмної інженерії
(повна назва)

КОМПЛЕКСНИЙ КУРСОВИЙ ПРОЄКТ
Пояснювальна записка

рівень вищої освіти _____ перший (бакалаврський)

Ігрове програмне забезпечення в жанрі соціального детективу на рушії Ren'Py

_____ (тема)

Виконав:

здобувач (ка) 3 курсу, групи ПЗПІ-23-3

Поліна ДАВИДЕНКО

_____ (Власне ім'я, ПРІЗВИЩЕ)

Спеціальність 121 – Інженерія програмного
забезпечення

_____ (код і повна назва спеціальності)

Тип програми освітньо-професійна

Освітня програма Програмна інженерія

_____ (повна назва освітньої програми)

Керівник ст.викл. кафедри ПІ Юрій НОВІКОВ

_____ (посада, Власне ім'я, ПРІЗВИЩЕ)

Члени комісії (Власне ім'я, ПРІЗВИЩЕ, підпис)

Ірина ГРУЗДО

Катерина ЗИБІНА

В'ячеслав ГРЕБЕНЮК

2026 р.

Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук _____
 Кафедра _____ програмної інженерії _____
 Рівень вищої освіти _____ перший (бакалаврський) _____
 Спеціальність _____ 121 – Інженерія програмного забезпечення _____
 Тип програми _____ Освітньо-професійна _____
 Освітня програма _____ Програмна Інженерія _____
 (шифр і назва)

Курс 3 _____ Група ПЗП-23-3 _____ Семестр 6 _____

ЗАВДАННЯ
на курсовий проект(роботу) студента

здобувачеві _____ Давиденко Поліні Ігоріні _____
 (прізвище, ім'я, по батькові)

1. Тема роботи _____ Ігрове програмне забезпечення в жанрі соціального детективу на рушії Ren'Py _____

2. Термін здачі студентом закінченої роботи „01” _____ червня _____ 2026 р.

3. Вихідні дані до проєкту

Методичні вказівки до виконання курсової роботи, специфікація програми _____

4. Перелік питань, що потрібно опрацювати в роботі

Вступ, аналіз предметної галузі, постановка задачі, архітектура та проєктування програмного забезпечення, опис прийнятих програмних рішень, аналіз отриманих результатів, впровадження програмного забезпечення, висновки, додатки.

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів роботи	Термін виконання етапів роботи	Примітка
1	Аналіз предметної галузі	18.03.2026	виконано
2	Розробка постановки задачі	02.04.2026	виконано
3	Проектування ПЗ	09.04.2026	виконано
4	Програмна реалізація	15.04.2026	виконано
5	Аналіз результатів	17.05.2026	виконано
6	Підготовка пояснювальної записки.	18.05.2026	виконано
7	Перевірка на наявність ознак академічного плагіату	31.05.2026	виконано
8	Захист роботи	03.06.2026	виконано

Дата видачі завдання “ 18 ” березня 2026р.

Здобувач  Поліна ДАВИДЕНКО
(підпис)

Керівник роботи _____ ст.викл. кафедри ПІ Юрій НОВІКОВ
(підпис) (посада, Власне ім'я, ПРІЗВИЩЕ)

РЕФЕРАТ / ABSTRACT

Пояснювальна записка містить: 81 стор., 48 рис., 10 джерел.

ВІЗУАЛЬНА НОВЕЛА, ДЕТЕКТИВ, ІГРОВЕ ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ, ІНТЕРАКТИВНІ ВИБОРИ, МІНІ-ІГРИ, СОЦІАЛЬНИЙ ДЕТЕКТИВ, REN'PY, PYTHON

Об'єкт розробки – ігрове програмне забезпечення в жанрі соціального детективу з елементами візуальної новели та інтерактивних міні-ігор.

Мета розробки – проектування та реалізація ігрового програмного забезпечення на рушії Ren'Py, яке поєднує нелінійний сюжет, систему виборів, механіку підозр та міні-ігри.

Метод розробки програмного забезпечення – рушій Ren'Py та мова програмування Python. Для створення користувацького інтерфейсу та візуального оформлення використовуються засоби Ren'Py, 2D-графіка та елементи аніме й noir-естетики.

У результаті розробки спроектовано та реалізовано ігрове програмне забезпечення в жанрі соціального детективу з багатоваріантним розвитком сюжету, системою підозр, сімома кінцівками та інтерактивними міні-іграми, що впливають на сюжет та фінальний результат гри.

VISUAL NOVEL, DETECTIVE, GAME SOFTWARE, INTERACTIVE CHOICES, MINI-GAMES, SOCIAL DETECTIVE, REN'PY, PYTHON.

The object of development is game software in the social detective genre with elements of a visual novel and interactive mini-games.

The purpose of the development is the design and implementation of game software using the Ren'Py engine, which combines a nonlinear storyline, a choice system, a suspicion mechanic, and mini-games.

The development method of the software is based on the Ren'Py engine and the Python programming language. Ren'Py tools, 2D graphics, and anime and noir aesthetic elements are used to create the user interface and visual design.

As a result of the development, game software in the social detective genre with a branching storyline, a suspicion system, 7 endings, and interactive mini-games that affect the plot and the final outcome of the game has been designed and implemented.

ЗМІСТ

Перелік скорочень	8
Вступ.....	9
1 Аналіз предметної галузі	10
1.1 Аналіз предметної галузі.....	10
1.2 Аналіз аналогів програмного забезпечення	11
1.3 Виявлення та вирішення проблем	14
2 Постановка задачі.....	16
3 Архітектура та проєктування програмного забезпечення	18
3.1 UML проєктування ПЗ.....	18
3.2 Проєктування архітектури ПЗ	20
3.3 Проєктування структури зберігання даних.....	22
3.4 Приклади використаних алгоритмів та методів.....	24
4 Опис прийнятих програмних рішень	26
4.1 Реалізація користувацького інтерфейсу	26
4.2 Реалізація головного меню та навігації.....	27
4.3 Реалізація сюжетної системи та діалогів	31
4.4 Реалізація системи підозр.....	33
4.5 Реалізація міні-ігор	34
4.5.1 Міні-гра «Втеча»	34
4.5.2 Міні-гра «Пошук предметів»	36
4.5.3 Міні-гра «Сліди на місці злочину».....	38
4.5.4 Міні-гра «Вітер брехні».....	40
4.6 Реалізація системи збереження та persistent-даних.....	41
5 Аналіз отриманих результатів	43
5.1 Порівняння результатів із початковими вимогами	43
5.2 Оцінка якості роботи системи	44
5.3 Аналіз ефективності прийнятих рішень	45
5.4 Обмеження та недоліки	46
6 Впровадження програмного забезпечення	47

Висновки	48
Перелік джерел посилання	49
Додаток А	50
Додаток Б.....	51
Додаток В	58
Додаток Г	74

ПЕРЕЛІК СКОРОЧЕНЬ

2D – двовимірна графіка

3D – тривимірна графіка

GDD – документ проєктування гри (Game Design Document)

GUI – графічний інтерфейс користувача (Graphical User Interface)

NPC – неігровий персонаж (Non-Player Character)

PNG – формат растрових графічних зображень (Portable Network Graphics)

RPY – сценарний файл рушія Ren'Py

UI – інтерфейс користувача (User Interface)

UML – уніфікована мова моделювання (Unified Modeling Language)

ВСТУП

Сучасні ігри все рідше зводяться до лінійного проходження. Гравець хоче впливати на події, тому жанр, де його вибір змінює поведінку персонажів і веде до різних фіналів, відповідає цьому запиту краще за інших.

Соціальний детектив як жанр побудований на недовірі – гравець не знає, хто бреше, і змушений аналізувати кожне слово. Саме це поєднання психологічної напруги й прийняття рішень визначило вибір теми. Формат візуальної новели тут підходить органічно: він концентрує увагу на діалогах і характерах, не перевантажуючи зайвими механіками. Міні-ігри доповнюють це – вони не дають грі перетворитися на пасивне читання з вибором відповіді наприкінці.

Актуальність роботи пов'язана зі зростанням інтересу до сюжетно-орієнтованих ігор та попиту на програмні продукти з нелінійним проходженням.

Метою роботи є проектування та реалізація гри в жанрі соціального детективу на рушії Ren'Py з використанням Python, системи виборів, механіки підозр та інтерактивних міні-ігор.

Результати роботи можуть бути використані як приклад розробки інтерактивного ігрового продукту з нелінійним сюжетом, а також як база для подальшого розширення – додавання нових гілок, персонажів і механік.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

1.1 Аналіз предметної галузі

Сюжетно-орієнтовані ігри давно вийшли за межі простої розваги – гравець у них не спостерігач, а учасник. Вибір, який він робить, змінює ставлення персонажів, відкриває або закриває сюжетні гілки, веде до різних фіналів. Саме це робить такі ігри цікавими для повторного проходження.

Соціальний детектив – один із найбільш вимогливих жанрів у цьому плані. Гравець мусить стежити за кожним діалогом, шукати суперечності, збирати докази й приймати рішення без повної інформації. Психологічна взаємодія між персонажами тут не фон – вона і є головним інструментом розслідування.

Міні-ігри в таких проєктах часто стають слабким місцем. Їх додають для динаміки, але реалізують як окремі вставки, що ніяк не впливають на сюжет. Через це гравець сприймає їх як перешкоду між діалогами, а не як частину історії. У дослідженнях динамічних наративних систем наголошується, що результати дій користувача мають змінювати внутрішній стан системи й активувати нові події – тільки тоді механіки й наратив працюють разом [1].

Нелінійність – одна з головних переваг жанру. Розгалужений сюжет і альтернативні кінцівки дають причину повернутися до гри після першого проходження. Не менш важливою є емоційна прив'язаність до персонажів, яка формується через діалоги, атмосферу й систему морального вибору.

Водночас є й очевидні складнощі. Великий обсяг тексту сповільнює темп, якщо інтерактивні механіки трапляються занадто рідко. Знайти баланс між сюжетом і геймплеєм непросто: надто багато механік відволікають від історії, а їх повна відсутність перетворює гру на читання з вибором відповіді.

Інтерфейс у таких іграх має залишатися мінімальним. Основні елементи – персонажі, текстове вікно, варіанти вибору. Для міні-ігор можна додавати таймери або шкали прогресу, але без перевантаження екрану. Інтерфейс повинен підтримувати атмосферу, а не руйнувати її.

З огляду на це, системи вибору, інтегровані міні-ігри й нелінійний сюжет є ключовими складовими жанру. Для визначення конкретних підходів до реалізації варто розглянути існуючі аналоги.

1.2 Аналіз аналогів програмного забезпечення

«Doki Doki Literature Club» – психологічна візуальна новела від Team Salvato (див. рис. 1.1) [2]. Гравець читає діалоги, взаємодіє з персонажами та іноді робить вибір. Головне, що тут працює – емоційна прив'язаність до героїв, яка потім використовується проти гравця. Атмосфера змінюється поступово, і саме це тримає увагу.



Рисунок 1.1 – Геймплей гри «Doki Doki Literature Club» (скриншот виконано самостійно)

Переваги:

- сильна психологічна атмосфера та емоційний вплив;
- виразні персонажі з опрацьованими діалогами;
- мінімалістичний інтерфейс, який не заважає сприйняттю;
- напруга наростає поступово – без різких стрибків.

Недоліки:

- інтерактивних механік майже немає;
- більшість виборів практично не впливає на сюжет;
- міні-ігор або складніших систем немає взагалі;
- переважання тексту подекуди знижує темп.

«Ace Attorney» від Capcom – детективна новела, де гравець виступає адвокатом. Потрібно збирати докази, допитувати свідків і знаходити суперечності у свідченнях прямо під час судових засідань. Механіки тут безпосередньо пов'язані із сюжетом – без доказів далі не пройти.



Рисунок 1.2 – Геймплей гри «Ace Attorney» (за даними [3])

Переваги:

- механіки розслідування вбудовані в сюжет, а не існують окремо;
- система аналізу свідчень дає відчуття реального детективного процесу;
- швидкий темп діалогів тримає увагу;
- персонажі запам'ятовуються.

Недоліки:

- сюжет здебільшого лінійний, варіативності кінцівок майже немає;
- деякі логічні рішення неочевидні без підказок;
- між інтерактивними сценами буває багато тексту;
- окремі механіки повторюються від епізоду до епізоду.

«Danganronpa» від Spike Chunsoft – детективна новела з ізольованою групою персонажів, серійними вбивствами та судовими розборами. Гравець досліджує локації, збирає докази й бере участь у судових засіданнях із міні-іграми. Жанровий мікс тут щільний: психологічний трилер, соціальний детектив і аркадні елементи в одному флаконі.

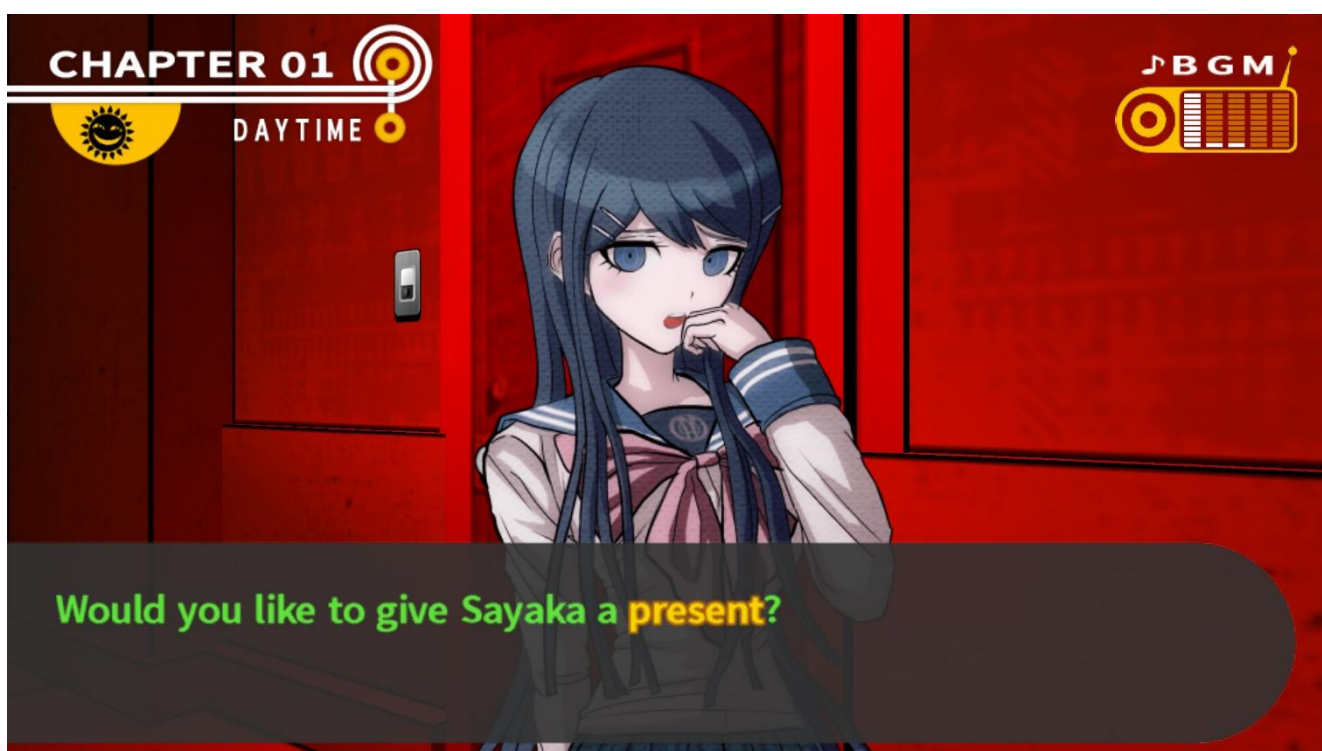


Рисунок 1.3 – Геймплей гри «Danganronpa» (за даними [4])

Переваги:

- міні-ігри вбудовані в ключові сцени, а не вставлені між ними;
- велика кількість персонажів із різними характерами та мотивами;
- емоційна напруга не спадає протягом більшої частини гри;
- сюжет постійно підкидає несподіванки.

Недоліки:

- деякі міні-ігри відволікають від основної історії;
- інтерфейс у окремих сценах перевантажений;
- новим гравцям частина механік дається важко;
- темп подачі подій інколи надто швидкий – важко встигнути за інформацією.

1.3 Виявлення та вирішення проблем

Аналіз предметної галузі й існуючих аналогів виявив кілька проблем, які стабільно повторюються в сюжетно-орієнтованих детективних іграх.

Перша проблема – це брак інтерактивності. Більшість таких ігор зводить взаємодію до читання діалогів і вибору реплік. Навіть популярні проекти рідко виходять за ці межі, а інтерактивні механіки з'являються епізодично. Під час довгих сюжетних сцен без жодної дії гравець просто відключається. Це дуже помітно в детективному жанрі, де напруга має підтримуватися постійно, а не лише в ключових моментах.

Друга проблема – міні-ігри, які живуть окремо від сюжету. Їх додають для динаміки, але результат найчастіше ні на що не впливає. Наратив йде своїм шляхом, міні-гра – своїм. У дослідженнях інтерактивних наративних систем підкреслюється, що дії користувача мають змінювати стан наративу і відкривати нові події – тільки тоді механіки і історія працюють разом [5].

Третя проблема – баланс між сюжетом і геймплеєм. Занадто багато механік – і гравець відволікається від історії. Занадто мало – і гра перетворюється на пасивне читання. Потрібні механіки, які не ламають темп і не виглядають вставками. Оптимальний підхід – інтегрувати їх у сюжетно значущі моменти, щоб геймплей і наратив підсилювали один одного.

Четверта проблема – це обмежена варіативність. Вибір у більшості таких ігор мало на що впливає: сюжет і без того йде до заздалегідь визначеного фіналу. Це знижує бажання проходити гру вдруге. Сучасні підходи до інтерактивних

систем вирішують це через механіки станів, параметри персонажів і умовні переходи між подіями [5].

П'ята проблема – перевантажений інтерфейс. Детектив як жанр вимагає одночасно тримати в голові діалоги, докази, підказки і механіки. Якщо все це ще й загромождає екран, то гравець губиться. Інтерфейс повинен допомагати орієнтуватися, а не змагатися з атмосферою за увагу.

Ці п'ять проблем і визначили вимоги до проєктованої системи. Кожна з них врахована при виборі механік, структури сюжету та організації інтерфейсу розроблюваної гри.

2 ПОСТАНОВКА ЗАДАЧІ

Мета роботи – розробка ігрового програмного забезпечення в жанрі соціального детективу на рушії Ren'Py [6] з використанням Python. За жанром це візуальна новела з системою виборів, механікою підозр та інтерактивними міні-іграми.

Сюжет будується навколо розслідування вбивства у віддаленому маєтку. Кожен персонаж щось приховує, у кожного є мотив. Гравець – один із гостей: ходить локаціями, розмовляє з підозрюваними, збирає інформацію та приймає рішення. Рішення впливають на перебіг подій і визначають фінал. Рівень підозри щодо кожного персонажа змінюється залежно від дій гравця та того, як він проходить міні-ігри.

Ігрові механіки та функціонал:

- реалізація системи діалогів із можливістю вибору реплік та дій;
 - реалізація сюжетних гілок із різними варіантами розвитку подій;
 - реалізація системи підозр для персонажів та гравця;
 - реалізація системи кінцівок залежно від кількості набраних підозр;
 - створення міні-гри на реакцію з натисканням випадкових клавіш у обмежений час;
 - створення міні-гри з пошуком прихованих об'єктів та системою підказок;
 - створення міні-гри з пошуком аномалій на зображенні;
 - створення міні-гри на психологічну взаємодію з персонажем через вибір реплік під час діалогу;
 - створення системи збереження та завантаження прогресу гри;
 - реалізація галереї відкритих кінцівок;
 - реалізація системи умовних переходів між подіями та сценами;
 - реалізація системи повторного проходження різних сюжетних гілок.
- Візуальний дизайн та інтерфейс користувача:
- створення 2D-візуального оформлення у стилі anime noir;
 - створення головного меню гри;
 - створення екрана гри з текстовим вікном та спрайтами персонажів;

- створення екрана вибору реплік і дій;
- створення інтерфейсів міні-ігор;
- створення меню збереження та завантаження гри;
- створення меню налаштувань звуку та режиму відображення;
- створення системи емоцій персонажів;
- створення візуального оформлення інтерактивних сцен та міні-ігор.

Тестування та оптимізація:

- функціональне тестування сюжетних переходів та системи вибору;
- перевірка коректності роботи системи підозр;
- тестування умов отримання кінцівок;
- перевірка роботи системи збереження та завантаження;
- тестування інтерфейсів та міні-ігор;
- перевірка стабільності роботи гри на desktop-платформах.

Технічна архітектура:

- розробка гри на рушії Ren'Py із використанням Python;
- розподіл проєкту на окремі файли для сюжету, інтерфейсу та міні-ігор;
- реалізація окремих модулів для роботи з графічним інтерфейсом, аудіо та трансформаціями;
- організація міні-ігор як окремих програмних компонентів;
- використання змінних для збереження стану проходження та результатів виборів;
- відтворення музики та звукових ефектів у межах ігрових сцен;
- забезпечення підтримки платформ Windows, macOS та Linux.

Буде створено концепт-документ гри з описом сюжетної структури, персонажів, механік та ігрових систем – його наведено у додатку В.

Підсумковий продукт – програмний застосунок у жанрі соціального детективу з повноцінними сюжетними механіками, системою підозр, міні-іграми та кількома варіантами розвитку подій.

3 АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 UML проєктування ПЗ

UML-діаграми у цьому проєкті – основний інструмент формалізації вимог і опису поведінки ігрового ПЗ. Для соціального детективу недостатньо просто перерахувати функції: важливо показати, як дії гравця змінюють стан сюжету, накопичують підозри й ведуть до конкретних кінцівок. Тому UML-моделювання охоплює і сценарії взаємодії з інтерфейсом, і внутрішню логіку обробки виборів та міні-ігор. У GDD гри зафіксовано, що основний геймплей будується на діалогах, системі підозр і вбудованих міні-іграх, а вибори прямо впливають на сюжет і фінальний результат проходження.

Перший рівень – діаграми варіантів використання. Ключовий актор взаємодіє з головним меню через кілька незалежних сценаріїв: запуск нової гри, завантаження збереження, доступ до галереї, налаштувань, довідки та вихід із застосунку. Отриману діаграму можна переглянути на рисунку Г.1 додатку Г.

Окремо виділені функції перегляду відкритих кінцівок, зміни гучності та перемикання повноекранного режиму.

Зв'язки між прецедентами побудовані ієрархічно. Запуск нової гри – базовий сценарій, завантаження прив'язане до вибору слоту збереження, що відповідає тому, як Ren'Py зберігає прогрес локально. Перегляд галереї дає доступ до вже відкритих кінцівок. Налаштування винесені окремо, бо не впливають на сюжет, але змінюють умови взаємодії з грою. Довідка виконує допоміжну роль – орієнтує гравця в базових механіках.

Наступна діаграма описує основний ігровий процес (див. рис. Г.2). Тут гравець взаємодіє вже не з меню, а безпосередньо з сюжетом: читає діалоги, обирає репліки, переходить між локаціями, запускає міні-ігри, зберігає прогрес, пропускає прочитані сцени та переглядає їх історію.

Важливий зв'язок у діаграмі – між вибором діалогу та запуском міні-ігор. Міні-гра не існує сама по собі: вона виникає як наслідок розвитку сюжету або конкретної сцени. Для соціального детективу це важливо – механіки мають підтримувати розслідування, а не відволікати від нього. Збереження гри теж є

ключовим прецедентом: проєкт передбачає кілька сюжетних гілок і повторне проходження, тому без можливості зберігатися гравець змушений проходити все заново.

Після опису функціональності – діаграми діяльності. Вони показують послідовність дій у межах сценарію та логіку переходів між станами. Діаграма діалогів і сюжетного потоку відображає базовий механізм проходження: запуск сцени, відображення тексту, вибір репліки, перевірка типу вибору, оновлення підозри, перехід до наступної сцени або окремої гілки. Отриману діаграму активностей можна переглянути на рисунку Г.3 у додатку Г.

Ключовий елемент діаграми – розгалуження між звичайним і важливим вибором. Звичайний продовжує діалог без суттєвого впливу на стан системи, важливий змінює рівень підозри. Обидва відкривають окремі сцени: частина виборів працює як атмосферний елемент, частина має прямі наслідки для фіналу. Перевірка умови досягнення кінцівки показує, що фінал визначається не одним рішенням, а загальним станом ігрової системи.

Діаграма діяльності для міні-ігор побудована окремо (див. рис. Г.4). Запуск переходить у перевірку результату, після чого система змінює один із параметрів підозри. Успіх збільшує підозру щодо конкретного NPC, невдача – підозру до самого гравця, після чого система повертається до основної сцени. Міні-ігри не є автономною розвагою всередині застосунку – вони працюють як частина детективної логіки та механізм розподілу підозр.

Для опису динамічної поведінки системи застосовано діаграми послідовностей. Перша діаграма (див. рис. Г.5) описує вибір репліки в діалозі та зміну підозри. Учасники: гравець, ігровий екран, сценарний файл і файл змінних. Гравець обирає відповідь, інтерфейс передає вибір у сценарну логіку, `script.rpy` визначає його тип. Якщо вибір важливий – оновлюється змінна підозри у `variables.rpy`, якщо звичайний – діалог продовжується без змін стану.

Значення, збережені у змінних, надалі використовуються при перевірці фінальних умов і відкритті гілок.

Друга діаграма послідовностей (див. рис. Г.6) описує запуск міні-гри та обробку її результату. Після виклику вступного екрана завантажується основний інтерфейс і починається цикл взаємодії: гравець натискає кнопки або клавіші, система оновлює стан і візуальні елементи. Після завершення перевіряється умова успіху або провалу: успіх збільшує підозру NPC, невдача – підозру гравця. Далі міні-гра ховається, сюжетна сцена відновлюється.

Міні-гра існує як окремий модуль інтерфейсу, але не як окремий режим застосунку. Її результат передається назад у сценарну частину й впливає на загальний стан проходження, тому гравець не відчуває розриву між сюжетом та активною взаємодією.

Загалом UML-моделювання охоплює і зовнішню взаємодію гравця з грою, і внутрішню логіку обробки сюжетних рішень. Діаграми варіантів використання формують функціональну картину системи, діаграми діяльності описують логіку переходів між сценами, діаграми послідовностей показують механізм оновлення стану під час діалогів і міні-ігор. Разом вони створюють основу для подальшого проєктування архітектури, структури збереження даних та алгоритмів реалізації.

3.2 Проєктування архітектури ПЗ

Архітектура гри побудована з урахуванням особливостей рушія Ren'Py та специфіки сюжетно-орієнтованих ігор. В основі – модульний підхід: окремі частини проєкту відповідають лише за свою область функціональності – сюжет, інтерфейс, міні-ігри, змінні стану та статичні ресурси.

Гра працює як desktop-застосунок без серверної частини. Уся логіка, ресурси, сценарії та збереження виконуються локально. Це відповідає концепції візуальної новели з нелінійним проходженням, де ключову роль відіграють сценарні переходи, зміна внутрішнього стану та система збережень.

Архітектура поділена на п'ять основних модулів: Narrative System, Minigames Module, GUI & Visuals, Core Engine та Static Resources.

Центральний елемент – Core Engine, який координує роботу решти компонентів. Тут розташовані основні сценарні файли: `script.rpy`, `variables.rpy` та

options.rpy. Файл script.rpy відповідає за розвиток сюжету, переходи між сценами, запуск діалогів та виклик міні-ігор. variables.rpy зберігає змінні стану – рівні підозр, прапори проходження, результати міні-ігор і умови відкриття кінцівок. options.rpy містить конфігураційні параметри: назву гри, налаштування відображення, параметри інтерфейсу та системні опції Ren'Py.

Narrative System реалізує сюжетну частину – файли персонажів і сценарні гілки. Цей компонент відповідає за побудову діалогів, розвиток сцен, взаємодію персонажів та переходи між гілками проходження. Кожен важливий вибір може змінити рівень підозри або вплинути на доступність наступних сцен, тому сюжетна система тісно пов'язана зі змінними стану.

Minigames Module містить інтерактивні механіки. Міні-ігри реалізовані як окремі сценарні модулі і викликаються безпосередньо із сюжетної частини. Їхня логіка ізольована від основного сценарію, що не порушує структуру проєкту. Результати передаються назад у Core Engine і впливають на загальний стан проходження через систему підозр.

Графічний інтерфейс і візуальна частина винесені в GUI & Visuals – screens.rpy, gui.rpy та transforms.rpy. Файл screens.rpy відповідає за екрани взаємодії: меню, діалогові вікна, систему збережень, налаштування та інтерфейси міні-ігор. gui.rpy визначає стилі елементів, шрифти, кольори та параметри відображення. transforms.rpy використовується для анімацій, переходів та трансформацій спрайтів.

Static Resources містить зображення, аудіофайли, шрифти та елементи графічного оформлення. Розділення ресурсів і сценарного коду спрощує структуру та полегшує оновлення контенту, а також важливе для роботи системи архівації Ren'Py. Компонентна структура проєкту наведена на рисунку Г.7.

Діаграма показує взаємозв'язок між підсистемами та роль Core Engine як центрального координатора. Narrative System і Minigames Module не звертаються до ресурсів напряму – доступ до стану гри та виклик інтерфейсів відбувається через основні сценарні файли. GUI & Visuals відповідає лише за відображення і не

містить сюжетної логіки, що не дає візуальній частині змішуватися з ігровими механіками.

Архітектура також враховує особливості виконання Ren'Py-застосунків на desktop-платформах. Основний вузол – пристрій користувача з Windows, macOS або Linux. Усередині запускається середовище виконання Ren'Py Runtime, яке обробляє сценарії, рендерить інтерфейс, відтворює аудіо та управляє внутрішніми системами рушія.

У межах середовища розміщується виконуваний файл гри MidnightDeception, який взаємодіє з директорією game/, бібліотеками lib/ та системними файлами рушія renpy/. Папка game/ містить сценарії, міні-ігри, графічний інтерфейс, ресурси та файли збережень. Директорія lib/ – бібліотеки й допоміжні залежності, renpy/ – внутрішні компоненти рушія.

Гра підтримує кілька сюжетних гілок і повторне проходження, тому файли збереження дозволяють швидко повернутися до потрібного моменту. Збереження створюються автоматично засобами Ren'Py і зберігаються локально на пристрої. Діаграма розгортання архітектури наведена на рисунку Г.8.

Модульне розділення дозволяє незалежно працювати із сюжетом, інтерфейсом і міні-іграми. Можливості Ren'Py при цьому спрощують реалізацію візуальної новели з розгалуженою структурою – без потреби будувати окремі системи для того, що рушій вже вміє з коробки.

3.3 Проектування структури зберігання даних

Для збереження стану гри використовуються вбудовані механізми Ren'Py. Оскільки проєкт – однокористувацька desktop-гра без мережевої взаємодії, усі дані проходження зберігаються локально. Це спрощує архітектуру та забезпечує швидке відновлення процесу без зовнішніх сервісів.

Основа структури даних – система змінних стану. Вона відстежує прогрес проходження, вибори користувача, рівні підозр персонажів та відкриті сюжетні гілки. Сценарна логіка звертається до цих змінних під час переходів між сценами, запуску діалогів, перевірки умов міні-ігор і визначення фінальних кінцівок.

Для кожного ключового персонажа зберігається окреме значення рівня підозри, яке змінюється залежно від дій гравця – під час діалогів, вибору реплік або завершення міні-ігор. Ці дані використовуються для перевірки умов фіналу та визначення результату розслідування.

Окремо зберігається сюжетний прогрес: активна сцена, поточна гілка проходження, виконані умови, результати попередніх виборів, відкриті діалогові переходи та доступність окремих сцен і кінцівок. Без цього нелінійна структура не працює – значна частина сцен стає доступною лише після виконання певних умов або накопичення конкретного рівня підозри.

Результати міні-ігор також фіксуються: успішність проходження, умови перемоги або поразки та їхній вплив на загальний стан. Після завершення міні-ігрової сцени результати передаються у сюжетну частину.

Механізм збереження Ren'Py автоматично фіксує поточний стан змінних під час створення save-файлу. Це дозволяє повністю відновити стан проходження, включаючи:

- поточну сцену;
- активний діалог;
- значення підозр;
- результати виборів;
- стан міні-ігор;
- відкриті сюжетні переходи.

Система слотів підтримує повторне проходження та дослідження різних гілок. Гравець може створювати кілька збережень у різних точках і повертатися до них, щоб змінити рішення.

Крім звичайних збережень, у проєкті використовуються persistent-дані Ren'Py. Вони зберігаються окремо від save-файлів і не видаляються після завершення проходження – використовуються для збереження відкритих кінцівок і роботи галереї. Гравець відкриває кінцівки без втрати вже отриманого прогресу.

3.4 Приклади використаних алгоритмів та методів

Логіка гри не спирається на складні математичні обчислення – вона тримається на постійному відстеженні стану проходження. Кожне рішення гравця щось змінює: відкриває нову сцену, коригує рівень підозри, впливає на фінал. Основні інструменти тут – умовні переходи, перевірка змінних і обробка подій.

Розгалуження сюжету реалізовано через систему умовних переходів між сценами. Перед кожним переходом перевіряються поточні значення змінних, і система визначає, яка сцена активується далі. Одна й та сама точка сюжету може вести в різні боки залежно від попередніх дій гравця.

Змінні стану і прапорці прогресу використовуються для перевірки виконаних умов, відкриття нових діалогів, запуску міні-ігор, визначення доступних варіантів відповіді і перевірки умов отримання кінцівки.

Підозра – головний параметр гри. Кожен ключовий персонаж і сам гравець мають власне значення, яке накопичується протягом усього проходження. Після важливих виборів або завершення міні-ігри система оновлює параметри: успішне проходження збільшує підозру щодо конкретного NPC, невдача – щодо самого гравця. Фінал визначається не одним вибором, а загальним станом усіх змінних на момент завершення гри.

Усі чотири міні-ігри мають спільну структуру: запуск окремої сцени, інтерактивна дія, перевірка умови, оновлення стану, повернення до сюжету. Відрізняється тільки механіка.

Міні-гра «Втеча» – механіка на реакцію: алгоритм генерує випадкові клавіші з випадковим положенням на екрані, паралельно відстежуючи правильність введення, кількість помилок і залишок часу.

Міні-гра «Вітер брехні» – механіка психологічного тиску: кожна репліка змінює параметр емоційного стану NPC, і після досягнення порогового значення відкривається можливість виявити суперечності у свідченнях.

Міні-гра «Пошук предметів» – механіка hidden object: алгоритм перевіряє натискання на визначені об'єкти сцени, правильна взаємодія оновлює прогрес, помилкова – скорочує таймер.

Міні-гра «Сліди на місці злочину» – механіка пошуку аномалій: алгоритм фіксує правильні та помилкові натискання і завершує сцену після досягнення потрібної кількості знайдених слідів або перевищення ліміту помилок.

Система збереження описана в підрозділі 3.3.

На етапі проектування визначено архітектуру застосунку, структуру даних і принципи роботи основних механік гри. Ці рішення лягли в основу програмної реалізації. Наступний розділ описує конкретні програмні рішення та особливості їх втілення.

4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ

Основні компоненти розроблюваної гри реалізовані на рушії Ren'Py з використанням Python. Проєкт розділений між окремими сценарними файлами так, щоб сюжетна логіка, інтерфейс і система збереження стану не перетиналися між собою. Це спрощує підтримку і дозволяє розширювати кожен частину незалежно.

4.1 Реалізація користувацького інтерфейсу

Інтерфейс гри побудований на системі screen-компонентів Ren'Py. Він охоплює головне меню, діалогові вікна, меню вибору реплік, збереження та завантаження, налаштування, історію діалогів і екрани міні-ігор.

Уся логіка відображення зосереджена у файлі `screens.rpy`. Кожен screen – окремий елемент, який викликається у потрібний момент сценарію: під час діалогу з'являється текстовий блок з іменем персонажа, при виборі репліки – меню з варіантами відповіді.

Візуальний стиль описаний у `gui.rpy`: кольори, шрифти, розміри елементів, стилі кнопок і параметри діалогового вікна. Усі ці параметри зібрані в одному місці, тому зміна оформлення не зачіпає сценарні файли.

Анімації реалізовані у `transforms.rpy`. Тут описані позиціонування спрайтів, ефекти тремтіння та масштабування.

Під час діалогу гравець бачить ім'я персонажа, текст репліки, спрайти, фон і варіанти відповіді. Варіанти можуть змінюватись залежно від попередніх рішень або результатів міні-ігор – це пов'язує інтерфейс із системою підозр і сюжетними змінними.

Додатково в системі діалогів використовується механізм `side image`, вбудований у Ren'Py. Під час відображення реплік поруч із текстовим блоком автоматично з'являється портрет персонажа, який у поточний момент веде розмову. Зображення автоматично оновлюється відповідно до активного персонажа та його емоційного стану (див. рис. 4.1).



Рисунок 4.1 – Екран гри під час діалогу (скріншот виконано самостійно)

Інтерфейси міні-ігор реалізовані як окремі screen-компоненти, що тимчасово замінюють діалогове вікно. Після завершення міні-гри система повертає гравця до основного сюжету без зміни режиму застосунку.

4.2 Реалізація головного меню та навігації

Головне меню гри реалізоване засобами screen-системи Ren'Py та виконує роль центральної точки навігації між основними функціями застосунку. Через нього користувач отримує доступ до запуску нової гри, завантаження збережень, налаштувань, галереї та виходу із застосунку.

Основна логіка меню реалізована через `screen main_menu()` (див. рис. 4.2). Для головного екрана використовується `imagemap`-підхід, при якому інтерактивні області накладаються поверх фонового зображення меню. Такий метод дозволяє створювати нестандартний інтерфейс без використання класичних кнопок Ren'Py та забезпечує точніше позиціонування елементів.



Рисунок 4.2 – Головний екран гри (скріншот виконано самостійно)

Фонове зображення меню використовується як основа інтерфейсу, а окремі стани кнопок реалізовані через idle- та hover-зображення. При наведенні курсора система автоматично змінює вигляд активної області, що створює ефект інтерактивності та покращує візуальне сприйняття меню.

Нижче наведено фрагмент реалізації screen головного меню:

```
screen main_menu():

    tag menu

    imagemap:
        ground "gui/main_menu.png"
        idle "gui/menu_normal.png"
        hover "gui/menu_hover.png"

        hotspot(1570, 458, 170, 79) action Start()
        hotspot(1570, 558, 160, 79) action ShowMenu("load")
    ...
```

У цьому фрагменті `imagemap` визначає систему інтерактивних зон поверх зображення меню. Команда `ground` задає основний фон, `idle` – стандартний вигляд кнопок, а `hover` – їх стан при наведенні курсора. Елементи `hotspot()` описують координати активних областей, на які може натискати користувач. Кожна область

пов'язана з окремою дією: запуском нової гри, відкриттям меню завантаження або переходом до інших screen-компонентів. Використання hotspot-областей дозволяє повністю контролювати розташування кнопок та створювати інтерфейс, який відповідає стилістиці гри.

Окрім головного меню, у грі реалізована система внутрішньої навігації через game menu (див. рис. 4.3). Вона використовується під час проходження та надає доступ до збереження гри, завантаження, історії діалогів, налаштувань, повернення до поточної гри, екрану допомоги, галереї, повернення до головного меню.

Game menu викликається без виходу з поточної сцени та не перериває основний ігровий процес. Це особливо важливо для сюжетно-орієнтованої гри, оскільки дозволяє користувачу швидко працювати із збереженнями або параметрами без втрати контексту сцени.

Окремо реалізована система галереї, яка використовується для перегляду відкритих кінцівок. Галерея працює на основі persistent-даних і зберігає інформацію про контент, відкритий під час попередніх проходжень.



Рисунок 4.3 – Меню гри (скріншот виконано самостійно)

Для побудови галереї використовується `screen gallery()`, у якому елементи розміщуються у вигляді сітки (див. рис. 4.4). Кожна клітинка відповідає окремій кінцівці або сцені.



Рисунок 4.4 – Галерея кінцівок гри (скріншот виконано самостійно)

Фрагмент реалізації галереї наведено нижче:

```
screen gallery():
    tag menu
    add "gui/game_menu.png"

    grid 3 3:
        xfill True
        yfill True

        add g.make_button("ending1", "police-small", xalign=0.5,
            yalign=0.5, hover_border="images/bg/gallery/hover.png")
        add g.make_button("ending2", "daniel_office-small",
            xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
        ...
```

У наведеному фрагменті `grid 3 3` використовується для створення сітки з дев'яти елементів. У середині сітки функція `g.make_button()` формує окремі кнопки галереї для відкритих кінцівок. Кожна кнопка містить власне зображення

попереднього перегляду та hover-ефект при наведенні курсора. Використання grid-структури забезпечує рівномірне розташування елементів незалежно від розміру екрана.

Для заблокованих кінцівок у галереї використовується окреме зображення-заповнювач. Якщо користувач ще не відкрив певний фінал, замість повноцінного preview-вікна відображається спеціальна картинка. Після отримання відповідної кінцівки система автоматично замінює заблокований елемент на справжнє зображення сцени.

Система навігації у розроблюваній грі побудована таким чином, щоб усі основні функції були доступні користувачу через єдину інтерфейсну структуру. Головне меню, game menu та галерея використовують спільний візуальний стиль і працюють через screen-механізми Ren'Py.

4.3 Реалізація сюжетної системи та діалогів

Сюжетна система гри реалізована засобами сценарної моделі Ren'Py. Основою побудови сюжету є система label, яка використовується для організації сцен, переходів між подіями та розгалуження сюжетних гілок. Кожен label представляє окрему сцену гри.

Перехід між сценами здійснюється через команди jump та call, які перенаправляють виконання сценарію до іншого label.

Для зміни локацій використовується команда scene. Вона очищує поточну сцену та завантажує новий фон, який відповідає іншому приміщенню або сюжетній ситуації.

Після зміни фону система відображає персонажів за допомогою команди show. Спрайти можуть змінюватися залежно від емоційного стану героя. Для одного персонажа використовується кілька варіантів виразів обличчя та поз.

Додатково під час відображення персонажів використовуються transforms і transition-ефекти. Вони відповідають за плавну появу спрайтів та зміну позиції персонажа на екрані.

Основою взаємодії користувача із сюжетом є система вибору реплік через конструкцію `menu`. У певних точках сценарію гравцю пропонуються кілька варіантів дій або відповідей, після чого система виконує відповідний перехід до нової сцени.

Нижче наведено приклад реалізації вибору у діалозі:

```
menu:
    'Run to Daniel`s office immediately.':
        jump go_to_office_fast

    'Wait for someone else to go first.':
        $ suspicion_mc += 1
        jump go_to_office_late

    'Try to leave the mansion.':
        jump try_escape
```

У наведеному фрагменті користувач отримує три варіанти дій. Кожен вибір переводить сюжет у різну гілку проходження через команду `jump`. Окремі варіанти можуть додатково змінювати внутрішній стан гри. У цьому прикладі другий вибір збільшує рівень підозри до головного героя через зміну значення `suspicion_mc`.

Для перевірки умов та відкриття окремих сюжетних гілок використовуються умовні конструкції. Система аналізує значення змінних стану перед запуском сцени або діалогу. Завдяки цьому одна й та сама сюжетна ситуація може розвиватися по-різному залежно від попередніх рішень користувача, результатів міні-ігор або рівня підозри персонажів.

Нижче наведено приклад перевірки умови переходу до фінальної сюжетної сцени:

```
if minigames_done >= 2:
    jump final_decision
```

У цьому фрагменті система перевіряє кількість завершених міні-ігор. Якщо користувач виконав необхідну умову, сценарій переходить до фінального етапу прийняття рішення.

Окрему роль у сюжетній системі відіграє механізм повторного проходження. Через використання save-файлів та persistent-даних користувач може повертатися до попередніх сцен і відкривати альтернативні варіанти розвитку подій.

Загалом сюжетна система гри побудована як набір взаємопов'язаних сцен із великою кількістю умовних переходів та змінних стану. Використання label-структури Ren'Py, системи menu-виборів і механізму jump-переходів дозволяє реалізувати нелінійний сюжет, у якому рішення користувача безпосередньо впливають на розвиток подій та фінальний результат проходження.

4.4 Реалізація системи підозр

Система підозр визначає результат розслідування і є основним інструментом розгалуження сюжету. Усі змінні підозр зберігаються у файлі `variables.rpy` і оновлюються протягом усього проходження.

Для кожного ключового персонажа передбачена окрема змінна; окремо відстежується підозра до самого гравця. Значення змінюються під час діалогів і після завершення міні-ігор.

Перед запуском окремих сцен і на етапі фінального вибору сценарій перевіряє поточні значення змінних.

Умови отримання кінцівок:

- кінцівка 1 – гравець утікає з маєтку, але подальший розвиток історії залишається невідомим, ця кінцівка відкривається після успішного проходження міні-гри «Втеча» на початку гри;

- кінцівки 2–4 – винним визнається один із персонажів: Енн, Софі або Віктор, конкретна кінцівка визначається залежно від того, до кого рівень підозри досягає 4 або більше;

- кінцівка 5 – винним визнається Майлз, якщо інші персонажі не набрали достатньої кількості підозр, а наприкінці саме його підставляють;

- кінцівка 6 – винним визнається гравець, відкривається, якщо не набрано достатньо підозр до інших персонажів, але Майлза не підставлено;

– кінцівка 7 – гравець залишається справжнім убивцею, але нікого не засуджують, ця кінцівка настає, якщо підозра до гравця досягає 5 або більше.

Система об'єднує діалоги, міні-ігри і сюжетні переходи в єдину структуру, де кожне рішення впливає на фінал.

4.5 Реалізація міні-ігор

Міні-ігри реалізовані як окремі screen-компоненти Ren'Py і запускаються безпосередньо зі сценарію. Після завершення кожна повертає гравця до основної сцени і передає результат у систему підозр. Для інтерактивності використовуються таймери, обробка клавіатурного вводу, перевірка умов і внутрішні змінні стану [7].

Перед запуском кожної міні-гри система викликає окремий intro-screen, у якому коротко пояснюється мета сцени та основні правила проходження. Такі екрани реалізовані через modal screen-компоненти Ren'Py та тимчасово блокують іншу взаємодію користувача до завершення вступної сцени.

4.5.1 Міні-гра «Втеча»

Міні-гра «Втеча» з'являється під час першого важливого розгалуження сюжету, коли гравець намагається залишити маєток одразу після вбивства. Основною механікою є гра на реакцію та швидкість введення (див. рис. 4.5).

Після запуску сцени система активує таймер і починає генерувати випадкові клавіші, які користувач повинен натискати у правильному порядку. Кожна правильна відповідь просуває проходження, а помилки накопичуються у внутрішньому лічильнику. Якщо гравець не встигає завершити потрібну кількість літер або перевищує допустиму кількість помилок, міні-гра завершується невдачею.

Для реалізації механіки використовується генерація випадкових літер через модуль random. Після створення нового символу система також випадково визначає його положення на екрані, що ускладнює проходження та змушує користувача постійно змінювати фокус уваги.

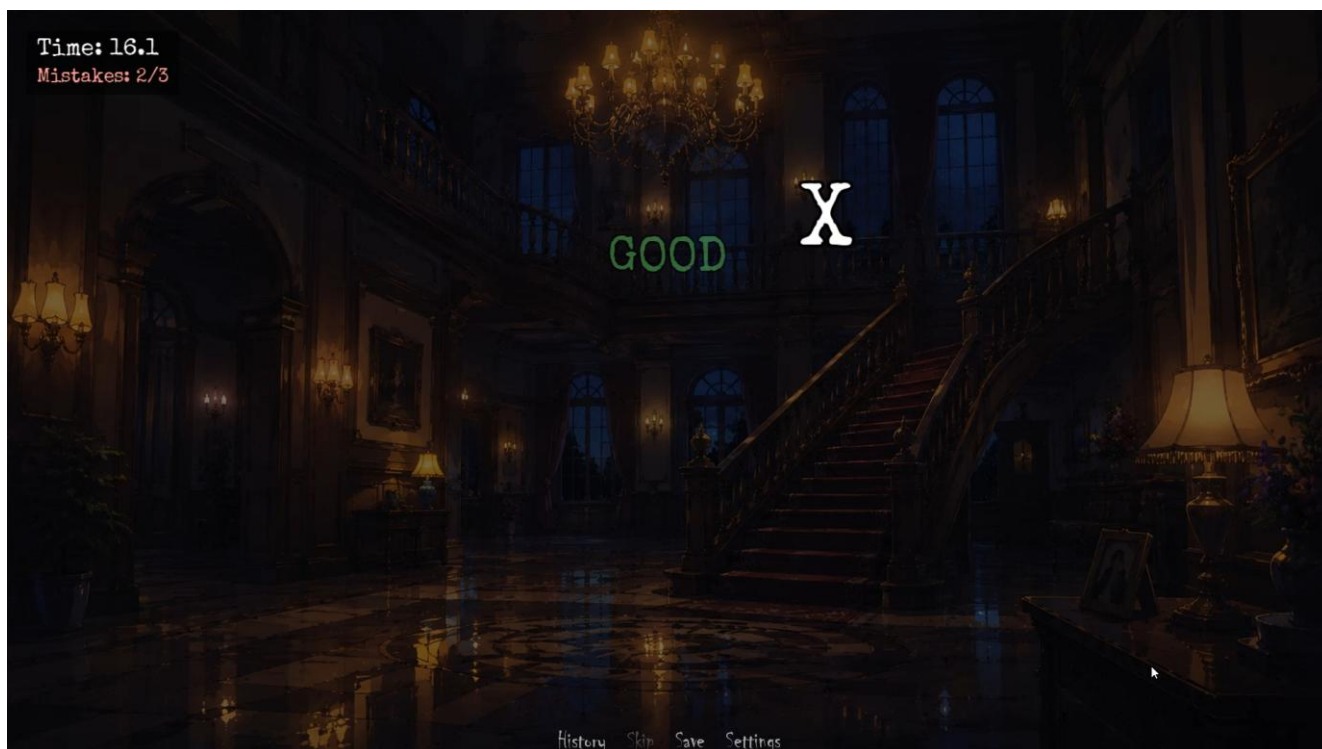


Рисунок 4.5 – Екран міні-гри «Втеча» (скриншот виконано самостійно)

Фрагмент генерації випадкової клавіші наведено нижче:

```
init python:
    import random
    import string

    def generate_escape_key():
        global escape_x
        global escape_y

        key = random.choice(string.ascii_uppercase)

        escape_x = random.uniform(0.1, 0.9)
        escape_y = random.uniform(0.1, 0.9)

        return key
```

У наведеному фрагменті функція `generate_escape_key()` створює випадкову літеру англійського алфавіту та одночасно задає нові координати її відображення на екрані. Завдяки цьому символ постійно змінює своє положення, що робить міні-гру динамічнішою.

Інтерфейс міні-гри реалізований через окремий `screen escape_game()`. На екрані одночасно відображаються:

- активна клавіша;
- таймер;
- кількість помилок;
- візуальні ефекти реакції на правильне або неправильне натискання.

Для посилення динаміки сцени використовуються transforms та screen-анімації. Під час гри застосовуються ефекти тремтіння екрана, короткі flash-повідомлення та анімації появи символів.

Ввід користувача реалізований через keyboard events Ren'Py. Система відслідковує натискання клавіш та перевіряє, чи відповідає введений символ поточному активному значенню.

У разі успішного проходження відкривається альтернативна кінцівка, у якій гравець залишає масток до розвитку основних подій. Якщо ж міні-гра завершується невдачею, користувач повертається до основного сюжету, а система додає +2 підозри до головного героя.

4.5.2 Міні-гра «Пошук предметів»

Міні-гра «Пошук предметів» відбувається в кабінеті Даніеля та використовується як частина розслідування. Основною метою сцени є пошук доказів, пов'язаних із персонажем Енн. Механіка побудована у форматі hidden object game, де користувач взаємодіє з інтерактивними предметами на статичному фоні (див. рис. 4.6) [8].

Сцена містить п'ятнадцять інтерактивних об'єктів, серед яких лише п'ять є важливими для проходження. Під час наведення курсора на предмет система підсвічує активну область та відображає короткий опис об'єкта.

До списку ключових предметів входять:

- флакон парфумів;
- шматок блискучої тканини;
- склянка з відбитком губної помади;
- амулет із знаком зодіаку;
- ключ від кабінету.

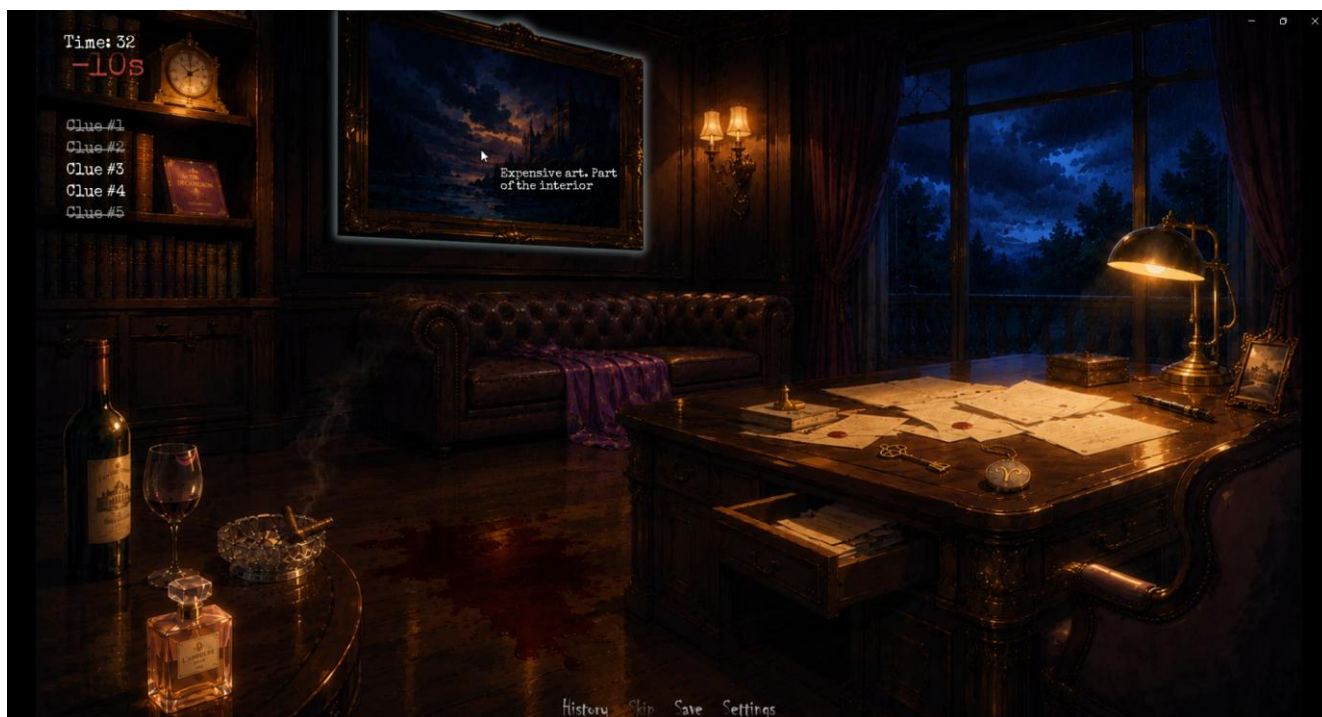


Рисунок 4.6 – Екран міні-гри «Пошук предметів» (скріншот виконано самостійно)

Для відображення опису знайденого предмета використовується окремий інформаційний блок, який з'являється поруч із курсором миші. Такий підхід дозволяє користувачу одразу отримати додаткову інформацію про об'єкт без переходу до окремого вікна або діалогової сцени. Позиція підказки автоматично оновлюється відповідно до поточного положення курсора.

Фрагмент реалізації наведено нижче:

```
frame at tooltip_fade:
xpos renpy.get_mouse_pos()[0] + 20
ypos renpy.get_mouse_pos()[1] + 20

background "#00000088"
xmaximum 220
```

У наведеному фрагменті координати інформаційного блоку визначаються на основі поточного положення курсора. Зміщення на 20 пікселів використовується для того, щоб підказка не перекривала сам об'єкт. Додатково застосовується `transform tooltip_fade`, який забезпечує плавну появу та зникнення

опису, а напівпрозорий фон покращує читабельність тексту незалежно від зображення сцени.

Після знаходження правильного об'єкта система автоматично викреслює його зі списку цілей.

Гра завершується автоматично після знаходження всіх потрібних предметів. Для ускладнення проходження використовується таймер. Якщо користувач натискає на неправильний об'єкт, система зменшує доступний час, що створює додатковий тиск під час пошуку.

Інтерактивність сцени реалізована через screen-компоненти Ren'Py та систему hotspot-областей. Кожен предмет має власну активну зону, яка перевіряє наведення курсора та натискання миші. Для підсвічування використовуються hover-ефекти, а стан знайдених предметів зберігається у внутрішніх змінних міні-гри.

Після завершення сцени система аналізує результат проходження. У разі успіху до Енн додається +2 підозри, оскільки всі знайдені докази пов'язані саме з нею. Якщо ж час закінчується до знаходження всіх предметів, гра завершується невдачею та додає +1 підозру до головного героя.

4.5.3 Міні-гра «Сліди на місці злочину»

Міні-гра «Сліди на місці злочину» використовується під час дослідження місця злочину та належить до категорії пошуку аномалій. На відміну від попередньої гри, де необхідно знаходити конкретні предмети зі списку, тут користувач повинен самостійно виявляти підозрілі деталі на зображенні та збирати докази, які можуть бути пов'язані з убивством (див. рис. 4.7).

Основою сцени є статичний фон приміщення, на якому розміщені інтерактивні області. Гравець повинен уважно оглянути локацію та знайти об'єкти, які вибиваються із загальної картини або можуть містити важливу інформацію для розслідування. Для підвищення складності частина об'єктів є малопомітною та органічно вписана в оточення.

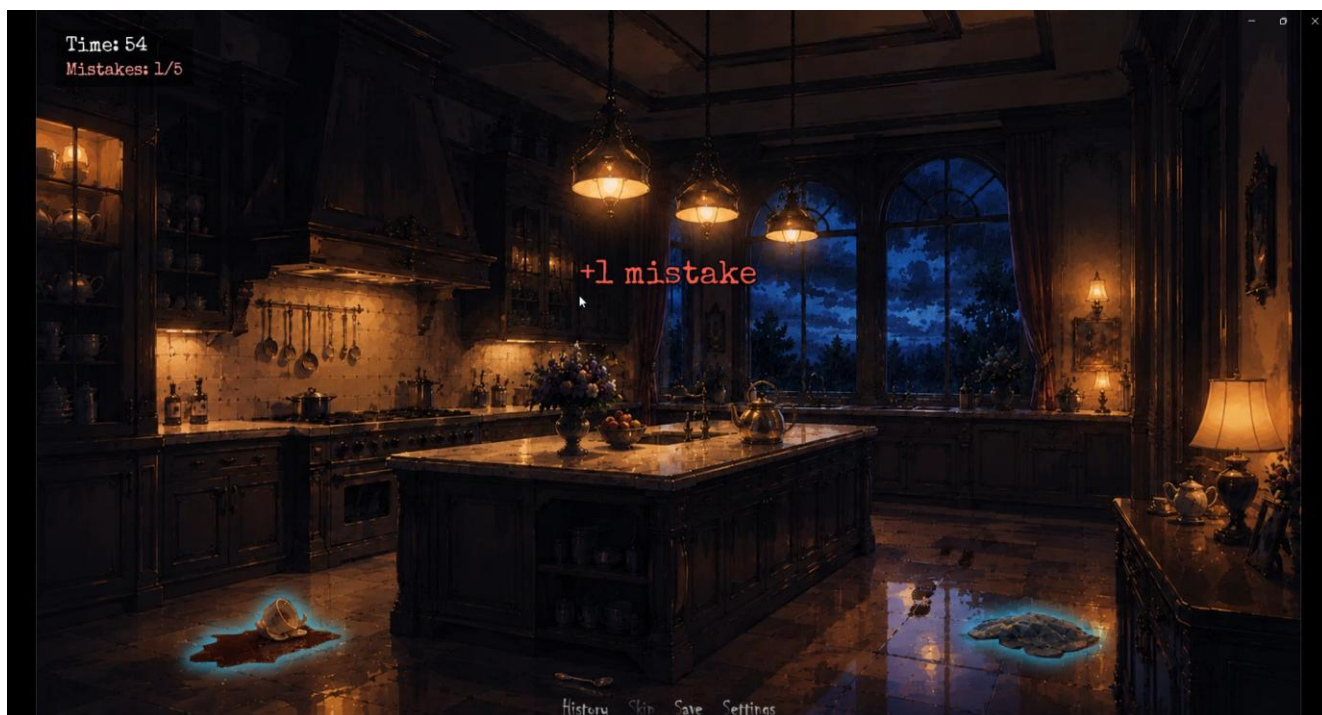


Рисунок 4.7 – Екран міні-гри «Сліди на місці злочину» (скріншот виконано самостійно)

Під час проходження система відстежує знайдені докази та кількість допущених помилок. Якщо користувач натискає на неправильну область, лічильник помилок збільшується. Одночасно на екрані відображається повідомлення про помилкову дію.

Інтерактивність реалізована через систему `imagebutton`, де кожен доказ має власну область взаємодії. Після знаходження потрібного об'єкта його стан змінюється, а система більше не дозволяє повторно зарахувати той самий доказ.

Фрагмент реалізації наведено нижче:

```
imagebutton:
if tea_found:
    idle "images/mg_anomaly/tea-hover.png"
else:
    idle "images/mg_anomaly/tea.png"
    action SetVariable("tea_found", True)
focus_mask True
```

У наведеному фрагменті змінна `tea_found` використовується для перевірки, чи був уже знайдений конкретний доказ. Якщо об'єкт ще не знайдено, користувач

може взаємодіяти з ним, після чого значення змінної змінюється на True. Після цього система відображає альтернативне зображення, яке сигналізує про виявлення доказу.

Параметр `focus_mask` True забезпечує більш точне визначення області натискання, оскільки взаємодія відбувається лише в межах видимої частини зображення, а не всієї прямокутної області кнопки.

Після завершення пошуку система перевіряє кількість знайдених слідів та визначає результат проходження. У разі успіху до Софі додається +2 підозри, оскільки всі знайдені докази пов'язані саме з нею. Якщо ж час закінчується до знаходження всіх предметів, або лічильник помилок переповнився, то гра завершується невдачею та додає +1 підозру до головного героя.

4.5.4 Міні-гра «Вітер брехні»

Міні-гра «Вітер брехні» використовується під час допиту Віктора. На відміну від інших міні-ігор, вона не перевіряє швидкість реакції або уважність гравця, а зосереджується на психологічній взаємодії з персонажем. Основною метою є поступове посилення емоційного тиску на співрозмовника для виявлення суперечностей у його свідченнях (див. рис. 4.8).

Під час проходження користувач отримує набір реплік, які можуть по-різному впливати на емоційний стан Віктора. Частина відповідей є нейтральною та практично не змінює перебіг сцени, тоді як інші створюють психологічний тиск і змушують персонажа нервувати. Завдання гравця полягає у правильному підборі реплік для поступового виведення співрозмовника з рівноваги.

Основою механіки є змінна `victor_emotion`, яка використовується для відстеження емоційного стану персонажа. Після кожної репліки система аналізує вибір користувача та відповідно змінює значення цієї змінної. Таким чином накопичується загальний рівень емоційного напруження протягом усієї сцени.



Рисунок 4.8 – Екран міні-гри «Вітер брехні» (скріншот виконано самостійно)

Після завершення міні-ігри система аналізує фінальний рівень `victor_emotion` та визначає результат проходження. Успішне проведення допиту збільшує підозру до Віктора. У разі невдачі персонаж зберігає самоконтроль, а користувач втрачає можливість отримати частину доказів у цій сюжетній гілці та отримує підозри до себе.

4.6 Реалізація системи збереження та persistent-даних

Для збереження прогресу використовуються вбудовані механізми Rep'Py. Екрани Save та Load доступні як із головного меню, так і під час проходження.

При створенні save-файлу рушій автоматично фіксує поточну сцену, активний діалог, значення підозр, результати міні-ігор і стан сюжетних прапорців. Після завантаження гра відновлюється точно в тому місці, де було зроблено запис.

Нелінійна структура сюжету робить систему збережень практично необхідною: гравець може зберегтися перед важливим рішенням і повернутися до нього, щоб дослідити іншу гілку без повторного проходження з початку.

Окремо використовуються persistent-дані. На відміну від звичайних save-файлів, вони не прив'язані до конкретного проходження і не видаляються при старті нової гри. У них зберігаються відкриті кінцівки, прогрес галереї і дані про вже переглянутий контент.

Саме persistent-дані забезпечують роботу галереї, описаної в підрозділі 4.2. Після отримання нового фіналу відповідний запис додається до глобального прогресу, і галерея автоматично розблоковує зображення. Нерозблоковані кінцівки відображаються як закриті.

Поєднання save-файлів і persistent-даних покриває обидві потреби: локальне збереження стану в межах поточного проходження і глобальний прогрес між різними сеансами гри.

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

5.1 Порівняння результатів із початковими вимогами

На етапі постановки задачі було визначено основні функціональні, інтерфейсні та технічні вимоги до розроблюваної гри. У результаті виконання роботи всі ключові вимоги були реалізовані та інтегровані в єдину програмну систему.

Серед функціональних вимог передбачалося створення нелінійної сюжетної структури з можливістю вибору реплік і дій. У розробленому програмному продукті реалізовано систему діалогів на основі сценарної моделі Ren'Py, що дозволяє користувачу впливати на розвиток подій та відкривати різні сюжетні гілки. Для підтримки варіативності проходження використано систему умовних переходів і змінних стану.

Однією з основних вимог була реалізація механіки підозр. Для кожного ключового персонажа та головного героя передбачено окремий показник підозри, який змінюється залежно від рішень користувача та результатів міні-ігор. На основі накопичених значень система визначає доступні кінцівки та результат розслідування. Таким чином, фінал гри залежить від сукупності дій, виконаних протягом проходження.

Також було поставлено завдання створити інтерактивні міні-ігри, інтегровані в сюжет. У результаті реалізовано чотири різні міні-ігри: «Втеча», «Пошук предметів», «Сліди на місці злочину» та «Вітер брехні». Кожна з них використовує власну механіку взаємодії та безпосередньо впливає на систему підозр і подальший розвиток подій. Завдяки цьому міні-ігри не є окремими вставками, а виступають складовою частиною процесу розслідування.

Вимоги до користувацького інтерфейсу також були виконані. Реалізовано головне меню, меню гри, систему налаштувань, екрани збереження та завантаження, історію діалогів, галерею відкритих кінцівок та окремі інтерфейси для кожної міні-ігри. Візуальне оформлення витримане в обраному стилі *anime noir*.

З технічної точки зору проєкт було реалізовано на рушії Ren'Py із використанням мови програмування Python. Архітектура побудована за модульним принципом із розділенням сюжетної логіки, інтерфейсу, міні-ігор та системи збереження даних. Для підтримки повторного проходження використано механізми save-файлів та persistent-даних, які забезпечують збереження поточного стану гри і глобального прогресу користувача.

Результати розробки відповідають вимогам, визначеним на етапі постановки задачі. Усі основні функціональні можливості, інтерфейсні компоненти та технічні рішення були реалізовані, що дозволило створити завершений програмний продукт у жанрі соціального детективу з нелінійним сюжетом та кількома варіантами проходження.

5.2 Оцінка якості роботи системи

Систему оцінювали за п'ятьма критеріями: коректність функціонування, стабільність, зручність використання, продуктивність і реіграбельність.

Тестування охопило всі основні механіки: сюжетні переходи, систему вибору реплік, механіку підозр, міні-ігри, збереження та завантаження прогресу. Всі ключові компоненти працюють коректно. Переходи між сценами виконуються за заданими умовами, зміни внутрішнього стану гри відображаються правильно впродовж усього проходження.

Критичних помилок під час перевірки не виявлено. Програма не завершується аварійно і не втрачає дані користувача. Система збереження фіксує поточний стан і коректно відновлює його після завантаження. Persistent-дані зберігають відкриті кінцівки та прогрес галереї між сесіями.

Інтерфейс відповідає жанровим особливостям візуальної новели. Елементи навігації розташовані там, де їх очікує користувач. Збереження, завантаження, налаштування та історія діалогів доступні прямо під час проходження. Єдиний візуальний стиль для меню, діалогових сцен і міні-ігор тримає інтерфейс цілісним.

Гра реалізована на Ren'Py і не використовує ресурсоємних обчислень або 3D-графіки, тому не потребує потужного обладнання. Сцени завантажуються, міні-ігри запускаються, екрани перемикаються без помітних затримок.

Реіграбельність забезпечується кількома сюжетними гілками, різними фіналами, системою підозр і галереєю кінцівок. Гравець має причину повернутися після першого проходження і дослідити альтернативні варіанти розвитку подій.

Загалом система стабільно виконує поставлені функції, не висуває значних вимог до заліза і підтримує повторне проходження завдяки нелінійній структурі сюжету.

5.3 Аналіз ефективності прийнятих рішень

Архітектурні та програмні рішення, прийняті під час розробки, були спрямовані на три речі: спростити реалізацію нелінійного сюжету, полегшити подальшу підтримку проєкту і виконати функціональні вимоги. Аналіз показав, що обрані підходи з цим впорались.

Вибір Ren'Py як рушія виявився виправданим. Вбудовані механізми діалогів, виборів, переходів між сценами, збережень та інтерфейсів скоротили обсяг допоміжного коду. Завдяки цьому основну увагу вдалося зосередити на сюжетній логіці, персонажах і міні-іграх.

Модульна архітектура спростила організацію проєкту. Розділення на окремі компоненти для сюжету, інтерфейсу, міні-ігор і змінних стану дозволяє змінювати одну частину без впливу на інші. Додавання нового функціоналу в майбутньому також не вимагатиме переписувати наявний код.

Система підозр стала основою нелінійного проходження. Окремі змінні для персонажів і головного героя об'єднали діалоги, міні-ігри та сюжетні переходи в єдиний механізм. На відміну від жорстко заданих сценаріїв, це забезпечує варіативність і визначає фінал на основі сукупності дій гравця.

Міні-ігри інтегровані безпосередньо в сюжет. У більшості подібних проєктів вони існують окремо і на історію не впливають. Тут результат кожної міні-гри змінює систему підозр і може вплинути на подальші події.

Persistent-дані Ren'Py дозволили реалізувати галерею кінцівок і зберігати глобальний прогрес між проходженнями. Гравець бачить, що ще не відкрив, і має причину повернутися.

Використання 2D-графіки та засобів Ren'Py знизило вимоги до заліза без втрати функціональності. Гра запускається на широкому спектрі комп'ютерів.

5.4 Обмеження та недоліки

Попри реалізацію основного функціоналу, проєкт має кілька обмежень.

Одним із головних недоліків є відсутність підтримки локалізації. Гра реалізована однією мовою, механізму перемикання між мовними пакетами не передбачено. Це звужує потенційну аудиторію.

Сюжетних гілок порівняно небагато. Гра підтримує кілька варіантів проходження і різні кінцівки, але кількість сценаріїв обмежена наявним контентом.

Озвучення персонажів відсутнє. Взаємодія відбувається через текст, звукові ефекти та фонову музику. Голосовий супровід міг би підсилити емоційне сприйняття і глибше занурити гравця в історію.

Гра орієнтована на desktop. Ren'Py підтримує мобільні платформи, але адаптація інтерфейсів міні-ігор і елементів керування для сенсорних пристроїв у цій роботі не виконувалася.

Система підозр працює на фіксованих порогових значеннях. Це спрощує реалізацію і робить поведінку системи передбачуваною, але обмежує глибину адаптації сюжету до дій гравця.

Міні-ігри мають обмежений набір механік і з'являються лише в окремих моментах. Їх кількість і різноманітність можна збільшити для підвищення інтерактивності.

Жодне з цих обмежень не впливає на коректність роботи основних механік і не заважає досягненню мети роботи. Але всі вони визначають напрями подальшого розвитку – локалізація, розширення контенту, мобільна адаптація і вдосконалення ігрових механік.

6 ВПРОВАДЖЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Гра призначена для локального використання на персональних комп'ютерах. Після завершення розробки проєкт збирається у виконувану версію засобами Ren'Py і поширюється у вигляді архіву. Щоб запустити гру, достатньо розпакувати архів у зручну теку і відкрити виконуваний файл. Жодних додаткових бібліотек чи серверних компонентів встановлювати не потрібно.

Для невеликих ігрових проєктів такий спосіб розповсюдження зручний: все просто, автономно, без залежностей. Гра не потребує підключення до мережі, всі дані зберігаються локально, а вбудована система збережень дозволяє відновлювати прогрес у будь-який момент.

Системні вимоги невисокі. Для коректної роботи достатньо комп'ютера з Windows, macOS або Linux, процесора базового рівня, 2 ГБ оперативної пам'яті та близько 500 МБ вільного місця на диску. Гра побудована на 2D-графіці та не навантажує залізо, тому запускається на більшості сучасних комп'ютерів.

Для поширення підходять сучасні цифрові платформи. Найдоступніший варіант для незалежних розробників – itch.io: публікація безкоштовна, підтримуються демоверсії та оновлення, є можливість збирати відгуки [9]. Для навчальних і інді-проєктів це оптимальний старт.

Для подальшого розвитку можлива публікація на Steam [10]. Платформа дає доступ до ширшої аудиторії, підтримує автоматичні оновлення, систему досягнень, статистику завантажень і взаємодію зі спільнотою. Розповсюджувати можна як безкоштовну, так і комерційну версію.

Готова збірка Ren'Py спрощує публікацію на обох платформах: рушій підтримує експорт під Windows, macOS та Linux без суттєвих змін у коді.

Загалом впровадження не вимагає складної підготовки з боку користувача. Гра запускається одразу після розпакування архіву, а невисокі системні вимоги і підтримка популярних платформ роблять її доступною для широкого кола гравців.

ВИСНОВКИ

У ході виконання курсової роботи було спроектовано та реалізовано ігрове програмне забезпечення в жанрі соціального детективу на рушії Ren'Py із використанням Python, системи виборів, механіки підозр та інтерактивних міні-ігор. Проєкт побудовано як нелінійну візуальну новелу, де рішення користувача впливають на розвиток подій, доступність окремих сюжетних гілок і фінальний результат проходження.

Створено сюжетну систему з діалогами, умовними переходами та змінними стану, які відстежують прогрес і рівень підозри до персонажів та самого гравця. Реалізовано окремі сюжетні гілки, систему кінцівок і механізм повторного проходження для дослідження альтернативних варіантів розвитку подій.

Чотири інтерактивні міні-ігри – «Втеча», «Пошук предметів», «Сліди на місці злочину» та «Вітер брехні» - не є автономними вставками. Кожна безпосередньо впливає на сюжет і систему підозр.

Для взаємодії з користувачем реалізовано головне меню, внутрішнє game menu, екрани вибору, галерею відкритих кінцівок, інтерфейси міні-ігор і систему збереження та завантаження прогресу. Persistent-дані зберігають відкриті фінали і забезпечують роботу галереї між різними проходженнями.

Архітектуру побудовано за модульним принципом із розділенням на сюжетну частину, інтерфейс, міні-ігри, змінні стану та статичні ресурси. Це спростило підтримку проєкту, зробило код структурованішим і заклало основу для подальшого розширення новими сценами, персонажами та механіками.

Отримані під час виконання роботи результати були частково представлено у тезах доповіді на науковій конференції [5].

Отримано завершений програмний продукт, який відповідає меті курсової роботи і поєднує цікавий сюжет, нелінійне проходження, інтерактивні елементи та механізми збереження прогресу в єдиній ігровій системі.

Репозиторій курсового проєкту разом із вихідним кодом:
https://github.com/NureDavydenkoPolina/2026_B_KKP_PZPI-23-3_Davydenko_P_I

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Шелл Д. Мистецтво геймдизайну / пер. Д. Цепкова. Київ : ArtHuss, 2025. 652 с.
2. Doki Doki Literature Club! : офіційний сайт. URL: <https://ddlc.moe/> (дата звернення: 22.03.2026).
3. Phoenix Wright: Ace Attorney Trilogy у Steam. URL: https://store.steampowered.com/app/787480/Phoenix_Wright_Ace_Attorney_Triology/ (дата звернення: 22.03.2026).
4. Danganronpa: Trigger Happy Havoc у Steam. URL: https://store.steampowered.com/app/413410/Danganronpa_Trigger_Happy_Havoc/ (дата звернення: 22.03.2026).
5. Давиденко П. І. Методи інтеграції міні-ігор у динамічні наративні системи комп'ютерних ігор // Пріоритетні напрямки та вектори розвитку світової науки : матеріали X Міжнар. студ. наук. конф., м. Кременчук, 24 квіт. 2026 р. Вінниця : ТОВ «УКРЛОГОС Груп», 2026. С. 138–139.
6. Ren'Py Visual Novel Engine : офіційний сайт. URL: <https://www.renpy.org/> (дата звернення: 25.03.2026).
7. Ess Ren'Py Tutorials : навчальний портал. URL: <https://essrenpytutorials.page/> (дата звернення: 25.03.2026).[
8. Coding With B and E. Ren'Py Tutorial Video. URL: <https://youtu.be/Ux7fNlXSLss> (дата звернення: 27.03.2026).
9. itch.io : платформа для інді-ігор. URL: <https://itch.io/> (дата звернення: 12.05.2026).
10. Steam Direct : офіційний портал розробників. URL: <https://partner.steamgames.com/steamdirect> (дата звернення: 12.05.2026).

ДОДАТОК А

Звіт результатів перевірки на унікальність тексту в базі ХНУРЕ

ДОДАТОК Б

Слайди презентації

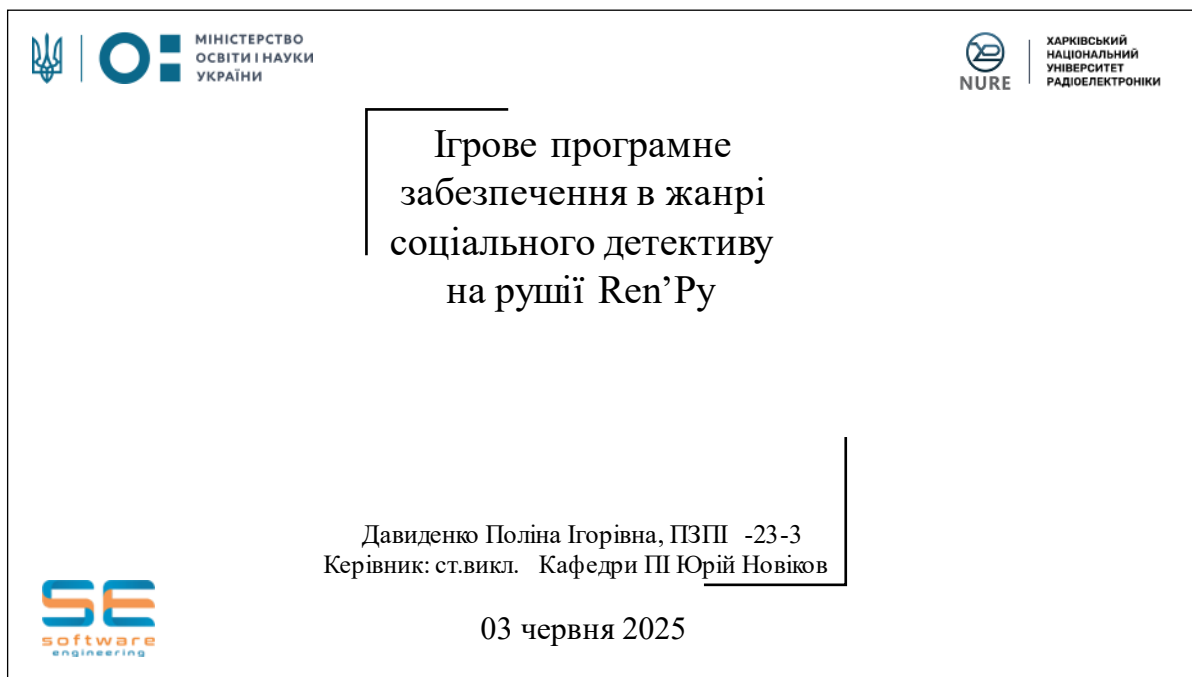


Рисунок Б.1 – Титульний слайд (рисунок виконаний самостійно)

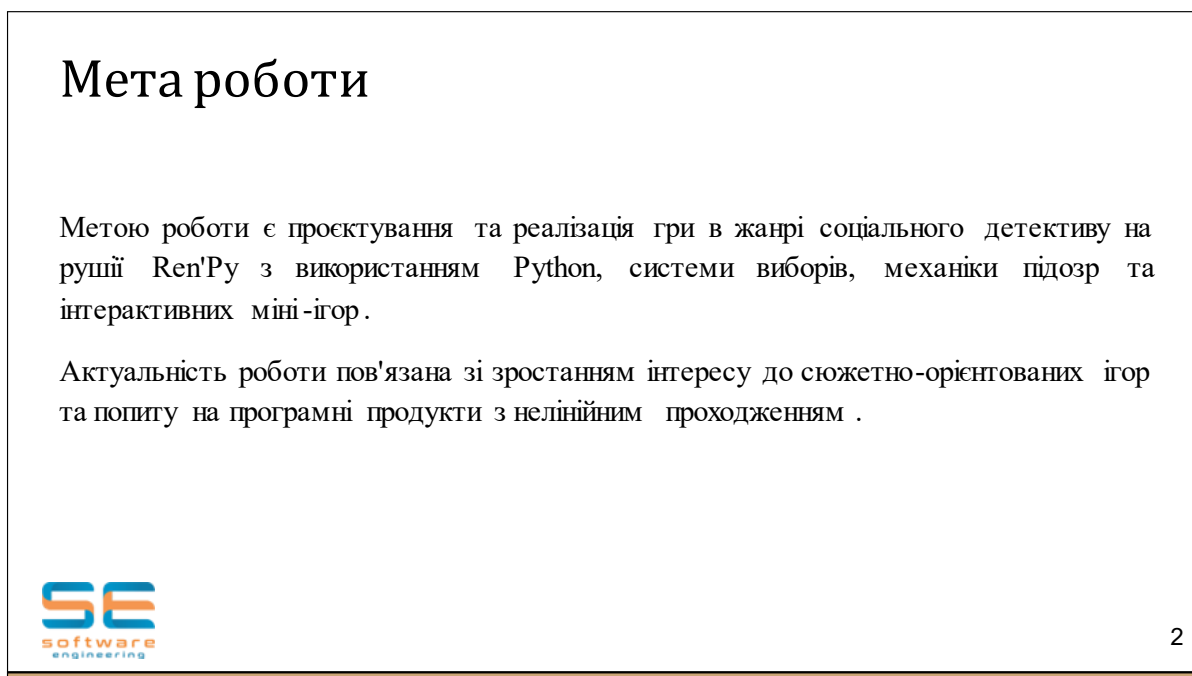


Рисунок Б.2 – Мета роботи та актуальність (рисунок виконаний самостійно)

Аналіз проблеми (аналіз існуючих рішень)

Аналог	Переваги	Недоліки
Doki Doki Literature Club	Сильна психологічна атмосфера, яскраві персонажі, мінімалістичний інтерфейс	Майже немає інтерактивних механік, вибори не впливають на основний сюжет
Ace Attorney	Механіки розслідування вбудовані в сюжет, швидкий темп, запам'ятовувані персонажі	Сюжет переважно лінійний, деякі логічні рішення неочевидні
Danganronpa	Міні-ігри пов'язані з ключовими сценами, багато персонажів, висока напруга	Інтерфейс перевантажений, окремі мінігри відволікають від історії



3

Рисунок Б.3 – Аналіз проблеми (аналіз існуючих рішень) (рисунок виконаний самостійно)

Постановка задачі та опис системи

У сюжетно-орієнтованих детективних іграх часто бракує інтерактивності, міні-ігри не впливають на сюжет, а інтерфейс може перевантажувати гравця.

Потрібно створити гру, де вибори та міні-ігри реально змінюють розвиток подій. Очікується створення нелінійної гри в жанрі соціального детективу на Ren'Py з системою виборів, підозр, 4 міні-іграми, кількома кінцівками, збереженням прогресу та галереєю відкритих фіналів.



4

Рисунок Б.4 – Постановка задачі та опис системи (рисунок виконаний самостійно)

Вибір технологій розробки

Ren'Py – рушій для розробки візуальних новел.

Python – реалізація логіки та міні-ігор.

Visual Studio Code – середовище розробки та редагування коду.

GitHub – контроль версій.



5

Рисунок Б.5 – Вибір технологій розробки (рисунок виконаний самостійно)

Архітектура створеного програмного забезпечення

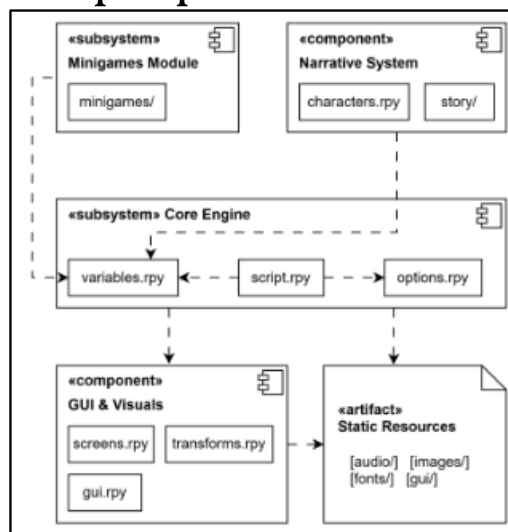
Core Engine – центральний модуль, що координує роботу всієї гри

Narrative System – сюжетні гілки, діалоги та система виборів

Minigames Module – реалізація інтерактивних міні-ігор.

GUI & Visuals – екрани, меню, анімації та інтерфейс користувача

Static Resources – графіка, аудіо та інші ресурси проекту.



6

Рисунок Б.6 – Архітектура створеного програмного забезпечення (рисунок виконаний самостійно)

Опис програмного забезпечення, що було використано у дослідженні

Ren'Py – основний рушій, використаний для створення візуальної новели, реалізації діалогів, меню, системи збережень та сюжетних переходів.

Python – використаний для програмування ігрової логіки, системи підозр та механік міні-ігор.

Visual Studio Code – середовище розробки для написання та редагування програмного коду проекту.

GitHub – використаний для зберігання проекту та контролю версій під час розробки.

Ren'Py Screen Language – застосований для створення користувацького інтерфейсу, меню, екранів діалогів і міні-ігор.



7

Рисунок Б.7 – Опис програмного забезпечення, що було використано у дослідженні (рисунок виконаний самостійно)

Дизайн системи

Методи: модульна архітектура, UML-моделювання, система умовних переходів, механіка підозр.

Послідовність: аналіз → проектування → реалізація → тестування.

Технології: Ren'Py, Python.



8

Рисунок Б.8 – Дизайн системи (рисунок виконаний самостійно)

Приклад реалізації

Галерея

```
screen gallery():
    top menu
    add "gui/game_menu.png"

    grid 3 3:
        xfill True
        yfill True

        add g.make_button("ending1", "police-small", xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
        add g.make_button("ending2", "daniel_office-small", xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
        add g.make_button("ending3", "kitchen-small", xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
        add g.make_button("ending4", "balcony-small", xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
        add g.make_button("ending5", "living_room-small", xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
        add g.make_button("ending6", "guest_room-small", xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
        add g.make_button("ending7", "house-small", xalign=0.5, yalign=0.5, hover_border="images/bg/gallery/hover.png")
```



9

Рисунок Б.9 – Приклад реалізації (галерея) (рисунок виконаний самостійно)

Приклад реалізації

Система вибору та сюжетних переходів

```
menu:
    'Run to Daniel's office immediately.':
        jump go_to_office_fast

    'Wait for someone else to go first.':
        $ suspicion_mc += 1
        jump go_to_office_late

    'Try to leave the mansion.':
        jump try_escape
```



10

Рисунок Б.10 – Приклад реалізації (Система вибору та сюжетних переходів)
(рисунок виконаний самостійно)

Приклад реалізації

Міні-гра «Пошук предметів»

```
frame at tooltip_fade:
xpos renpy.get_mouse_pos()[0] + 20
ypos renpy.get_mouse_pos()[1] + 20

background □ "#00000088"
xmaximum 220
padding (10, 8)

text screen.tooltip:
color □ "#fff"
size 18
```



11

Рисунок Б.11 – Приклад реалізації (Міні-гра «Пошук предметів») (рисунок виконаний самостійно)

Інтерфейс користувача



12

Рисунок Б.12 – Інтерфейс користувача (рисунок виконаний самостійно)

Підсумки

Розроблено гру у жанрі соціального детективу на рушії Ren'Py.

Реалізовано нелінійний сюжет, систему виборів та механіку підозр.

Створено 4 інтерактивні міні -ігри, інтегровані в сюжет.

Розроблено систему збереження прогресу та галерею кінцівок.



Рисунок Б.13 – Підсумки (рисунок виконаний самостійно)

ДОДАТОК В

Специфікація програмного забезпечення

ГДД "Midnight Deception"



1. Тетра
2. Введення
3. Цільова аудиторія
4. USP
5. Час ігрової сесії
6. Технічні характеристики
7. Взаємодія з грою
8. Опис гри
 - 8.1.Інтерфейси
 - 8.2.Ігровий процес
 - 8.3.Механіки
 - 8.3.1 Діалоги та вибори
 - 8.3.2 Система збережень
 - 8.3.3 Система підозр
 - 8.3.4 Міні-ігри
 - 8.3.5 Кінцівки
 - 8.3.6 Схеми гілок
 - 8.4.Аудіо

Рисунок В.1 – Сторінка 1 ГДД розроблюваної гри (скриншот виконаний самотійно)

1. Тетра

Технологія - ПК

Механіка - візуальна новела з міні-іграми

Історія - ГГ - може бути як детективом, так і прихованим убивцею

Естетика - аніме, poir

2. Введення

Ніч була холодна як для осені. Подвір'я великого будинку освітлювали ліхтарі, а за великими вікнами вже давно йшло невелике зібрання гостей.

Даніель, господар будинку, чоловік близько п'ятдесяти років, багатий і впевнений у собі, запросив цих людей ніби випадково. Але випадковості тут не було. У кожного гостя була своя причина прийти, а в нього — своя таємниця, прихована за ідеальною усмішкою.

Ти теж був серед гостей. Твоє ім'я було у списку, але ти не зовсім розумів, чому тебе запросили. З Даніелем ти спілкувався мало, а з іншими гостями практично не був знайомий. Мабуть, саме тому тобі було цікаво. Ти завжди ставив багато питань, завжди хотів знати більше, ніж здавалось потрібним. Ця цікавість то допомагала, то створювала проблеми.

У вітальні були всі гості будинку:

- Енн — молода жінка з проникливим поглядом. Вона говорила так, ніби бачить більше за інших. Любила натякати на долю й карму, але користувалася новим смартфоном і носила дорогі прикраси.

Рисунок В.2 – Сторінка 2 ГДД розроблюваної гри (скріншот виконаний самотійно)

- Софі — душа компанії. Весела, добра, сміх її піднімав настрій усім. Вона здавалася щасливою бути тут і намагалася піднімати настрій іншим.
- Віктор — чоловік середніх років, ввічливий і м'який. Хотів виглядати надійним, але його очі видавали хвилювання: він часто озирався і здавався тривожним.
- Майлз — похмура постать у кутку. Колючий погляд, саркастичні слова, руки схрещені на грудях — він не приховував, що може бути небезпечним.
- Даніель — господар будинку. Він ходив серед гостей із келихом вина, розповідав про бізнес, подорожі та багатство. Кожне його слово показувало впевненість і владу.
- Ти спостерігав, ставив питання — хтось відповідав охоче, хтось ухильно. Але відчувалося: за усмішками ховається щось більше.

І раптом усе змінилося. Гучний звук розірвав ніч. Спочатку ніхто не зрозумів, що сталося. Гості кинулися до кабінету — там лежав Даніель у крові. Його очі були широко відкриті, обличчя застигло у страху. Він був мертвий. Настала тиша, а потім паніка, крики і плутанина.

І саме тоді твоя цікавість заговорила голосніше: хто вбив Даніеля? І чому всі ці різні люди опинилися тут цієї ночі?

Від твоїх рішень залежало, хто виживе і хто виявиться вбивцею.

Рисунок В.3 – Сторінка 3 ГДД розроблюваної гри (скріншот виконаний самостійно)

3. Цільова аудиторія

- Вік: 16+ (через детективну тематику, напружену атмосферу та потенційно драматичні сцени).
- Інтереси: любителі інтерактивних історій, детективів, психологічних трилерів; фанати візуальних новел та анімешної естетики; люди, які люблять розгадувати загадки і впливати на сюжет.
- Досвід у іграх: від новачків до досвідчених гравців; механіка проста для освоєння, але сюжет та вибори заохочують переігрування.

4. USP

- Динамічний сюжет: вибір гравця впливає на кінцівку - у кожній сюжетній гілці різні персонажі можуть бути вбивцями, а сам гравець може опинитися серед підозрюваних чи самим вбивцею, також є можливість звинуватити невинного.
- Міні-ігри: інтерактивні завдання вплетені у сюжет, підсилюють детективну атмосферу; пошук доказів, допит — все пов'язано з розслідуванням і безпосередньо впливає на результати.
- Поєднання естетики: змішання аніме та нуар-стилю створює унікальну атмосферу напруженої історії з виразними персонажами.

Рисунок В.4 – Сторінка 4 ГДД розроблюваної гри (скріншот виконаний самотійно)

5. Час ігрової сесії

- Тривалість однієї сюжетної гілки: ~30 хвилин.
- Повне проходження всіх кінцівок: залежно від швидкості та повторного проходження — 2-4 години.
- Повторне проходження мотивоване: різні вибори відкривають нові гілки, різних вбивць та фінальні сцени.

6. Технічні характеристики

Гра розробляється для ПК із використанням рушія Ren'Py та мови Python, що дозволяє легко реалізувати візуальну новелу з гілками сюжету та міні-іграми. Гра працює на операційній системі Windows, Mac та Linux. Управління здійснюється за допомогою клавіатури та миші, що дозволяє обирати репліки, взаємодіяти з предметами та проходити міні-ігри. Виведення зображення відбувається на моніторі комп'ютера. Графіка — у форматі 2D із елементами аніме та нуар-естетики. Звукова система підтримує колонки ПК та навушники.

Зберігання даних гри здійснюється на жорсткий диск або твердотільний накопичувач. Інтернет-з'єднання для гри не потрібне, усі файли, сюжетні лінії та міні-ігри зберігаються локально. Гра підтримує систему збережень і завантажень прогресу, а також галерею пройдених сцен та кінцівок, що дозволяє гравцю перегравати різні гілки сюжету та відкривати нові фінали.

Рисунок В.5 – Сторінка 5 ГДД розроблюваної гри (скриншот виконаний самостійно)

7. Взаємодія з грою

Гравець взаємодіє з грою за допомогою клавіатури та миші. Всі дії поділяються на вибір діалогів, навігацію по тексті, проходження міні-ігор та управління меню.

Елемент	Дія	Клавіша / Кнопка
Вибір реплік	Вибір однієї з декількох варіантів відповіді або дії	Лівий клік миші на варіанті
Перехід до наступного тексту	Продовження діалогу або опису	Лівий клік миші на вікно діалогу; клавіша Пробіл ; клавіша Enter
Пропуск тексту	Швидке проходження діалогу, який уже читався	Кнопка Skip на екрані
Прискорення тексту	Виведення тексту швидше під час читання	Додатковий лівий клік миші
Повернутися назад	Повернути минулу репліку	Кнопка Back
Міні-ігри	Початок і проходження міні-гри	Лівий клік для взаємодії; додаткові клавіші залежать від типу міні-гри (стрілки, Пробіл , різні клавіші)
Меню паузи	Виклик головного меню гри, збереження/завантаження	Клавіша Esc або кнопка Prefs та Save на екрані

Рисунок В.6 – Сторінка 6 ГДД розроблюваної гри (скріншот виконаний самостійно)

8. Опис гри

8.1.Інтерфейси

Основні інтерфейси:

1. Головний екран – містить назву гри та основні елементи керування: кнопку для початку нової гри, можливість продовжити одну з раніше розпочатих, перехід до налаштувань, галерея, вихід із гри.
2. Екран гри – складається із заднього фону, зображень спрайтів персонажів, текстового вікна для діалогів чи описів та основних кнопок унизу по центру – меню та скіп для пропуску сцен.
3. Екран гри під час вибору варіанту відповіді чи дії – виглядає так само, як звичайний ігровий екран із фоном, персонажами та текстовим вікном, але посередині з'являються варіанти вибору, які можна обрати натиском лівої кнопки миші.
4. Міні-гра «Пошук предметів» відображає час та підказку до завдання у верхньому правому куті, а гравець мишею наводить курсор на предмети на екрані. При виділенні предмет підсвічується, з'являється його опис.
5. Меню – містить кнопки для збереження гри та відображає слоти з пустими або вже збереженими іграми. Щоб завантажити гру, потрібно обрати непустий слот і натиснути відповідну кнопку загрузити гру. Також є кнопки для повернення до головного екрану, відкриття налаштувань, виходу з гри та повернення до поточного ігрового екрану.
6. Налаштування – дозволяють регулювати гучність музики та звукових ефектів, а також вибирати, чи гра відображається у повноекранному режимі.

8.2.Ігровий процес

Гравець потрапляє у великий будинок як один із запрошених гостей і спостерігає за розвитком подій через текстові описи та діалоги. Основна взаємодія відбувається через вибір реплік і дій, що

Рисунок В.7 – Сторінка 7 ГДД розроблюваної гри (скріншот виконаний самостійно)

впливають на розвиток сюжету та ставлення інших персонажів. Під час гри гравець може досліджувати приміщення, знаходити предмети, ставити питання та збирати докази, які допомагають у розслідуванні вбивства Даніеля. Важливу роль відіграє увага до деталей: будь-яке рішення може змінити сюжетну гілку, визначити, хто виживе, хто виявиться вбивцею або чи опиниться сам гравець серед підозрюваних.

Динаміка гри поєднує сюжетні сцени з інтерактивними елементами - під час міні-ігор гравець виконує завдання, що безпосередньо впливають на результати розслідування. Управління здійснюється клавіатурою та мишею.

8.3.Механіки

Основна механіка гри «Midnight Deception» побудована на системі інтерактивної візуальної новели з гілками виборів, підозрами та вбудованими міні-іграми.

8.3.1 Діалоги та вибори

Ігровий процес рухається через послідовні сцени діалогу. Кожна репліка супроводжується спрайтами персонажів та текстом у вікні. У певні моменти з'являються варіанти вибору, від яких залежить напрямок сюжету і рівень підозр (про них в пункті 8.3.3).

Рисунок В.8 – Сторінка 8 ГДД розроблюваної гри (скріншот виконаний самотійно)

8.3.2 Система збережень

Гра підтримує збереження прогресу у будь-який момент. Після завантаження гра автоматично відтворює сцену з цього місця. Кожен слот має час, дату та зображення моменту на якому було збережено.

8.3.3 Система підозр

У грі реалізована система підозр, яка визначає хто врешті буде оголошений винним. Під час проходження різні вибори та результати міні-ігор впливають на рівень підозри кожного з персонажів.

Загальні правила:

- Показник підозри може змінюватися внаслідок вибору діалогу, дій або результатів міні-ігор.
- Максимальна кількість підозр у персонажа або гравця не обмежується, але кінцівки визначаються при досягненні певних порогів.

8.3.4 Міні-ігри

1. Міні-гра "Втеча"

Місце: перше розгалуження (після спроби втекти з будинку).

Тип: гра на реакцію.

Механіка: Після запуску сцени на екрані з'являється таймер. Система генерує випадкову послідовність клавіш (рандомні літери), які потрібно

Рисунок В.9 – Сторінка 9 ГДД розроблюваної гри (скриншот виконаний самостійно)

натискати у правильному порядку, поки час не вичерпається. Кожне правильне натискання продовжує рух сцени. Припустимо п'ять помилок.

Результати:

- Успіх → кінцівка 1: гравець втікає, поліція знаходить вбивцю.
- Невдача → +2 підозри до гравця, повернення до основного сюжету.

2. Міні-гра "Пошук предметів"

Місце: кабінет Даніеля.

Тип: пошук предметів.

Механіка: Сцена складається з одного фону з 15 інтерактивними об'єктами, потрібні 5. При наведенні курсора на предмет він підсвічується; клік запускає короткий опис. Кожен знайдений потрібний об'єкт видаляється зі списку завдань і додає очко до прогресу. Коли всі об'єкти знайдені - гра завершується автоматично. Причому при натиску на неправильний предмет, віднімається певний час. Предмети, які треба знайти:

- Флакон парфумів. При натисканні: "Той самий запах, що й від Енн..."
- Шматок блискучої тканини. При натисканні: "Ця тканина надто дорога, щоб належати служниці."

Рисунок В.10 – Сторінка 10 ГДД розроблюваної гри (скріншот виконаний самостійно)

- Склянка з відбитком губної помади. При натисканні: "Він пив не сам. Ця помада - точно не його стиль."
- Амулет із знаком зодіаку. При натисканні: "Вона ніколи не знімає його. Як він опинився тут?"
- Ключ від кабінету. При натисканні: "Міг належати господарю... або тому, хто його вкрав."

Результати:

- Успіх → +2 підозри до Енн (бо ми шукали речі саме пов'язані з нею).
- Невдача → +1 підозра до гравця (час сплив).

3. Міні-гра "Сліди на місці злочину"

Місце: кухня.

Тип: пошук аномалій.

Механіка: Система завантажує зображення локації з прихованими "аномаліями" (наприклад, відбитки, плями, зламані предмети). Кожна правильна знахідка додає очко до шкали прогресу. Є декілька спроб натиснути неправильно і не втратити результат.

Результати:

- Успіх → +2 підозри до Софі (бо саме її алібі на кухні).
- Невдача → +1 підозра до гравця (2+ неправильних натискань).

Рисунок В.11 – Сторінка 11 ГДД розроблюваної гри (скріншот виконаний самостійно)

4. Міні-гра "Вітер брехні"

Місце: балкон.

Тип: діалогова інтерактивна сцена.

Механіка: Під час діалогу гравець обирає тип відповіді на репліки персонажа, наприклад, "спокійно", "тиснути", "уникнути теми". Від вибору залежить емоційна реакція співрозмовника та можливість виявити суперечності. Кожен вибір впливає на шкалу емоційності персонажа. Коли шкала досягає порогового значення (5), персонаж починає видавати суперечності, і гравець отримує шанс викрити його.

Результати:

- Успіх → +2 підозри до Віктора.
- Невдача → +1 підозра до гравця.

8.3.5 Кінцівки

- Кінцівка 1 - Ми втікаємо, але не знаємо продовження історії.
Отримується, якщо на початку гри пройти міні гру «Втеча» успішно.
- Кінцівка 2 - Винна Енн.
Якщо за гру набрано 4+ очків підозри до Енн.
- Кінцівка 3 - Винна Софі.
Якщо за гру набрано 4+ очків підозри до Софі.

Рисунок В.12 – Сторінка 12 ГДД розроблюваної гри (скриншот виконаний самостійно)

- Кінцівка 4 - Винний Віктор.
Якщо за гру набрано 4+ очків підозри до Віктора.
- Кінцівка 5 - Винний Майлз (підставлено).
Якщо не набрано достатньо підозр до інших персонажів, але в кінці підставити Майлза.
- Кінцівка 6 - Винний гравець.
Якщо не набрано достатньо підозр до інших персонажів, але в кінці не підставити Майлза.
- Кінцівка 7 - Нікого не посадили, хоча гравець і є вбивцею.
Якщо гравець має 5+ очків підозри.

8.3.6 Схеми гілок

Повну схему можна переглянути за посиланням:

https://miro.com/app/board/uXjVJ_IsofE=?share_link_id=197958096326

Рисунок В.13 – Сторінка 13 ГДД розроблюваної гри (скриншот виконаний самостійно)

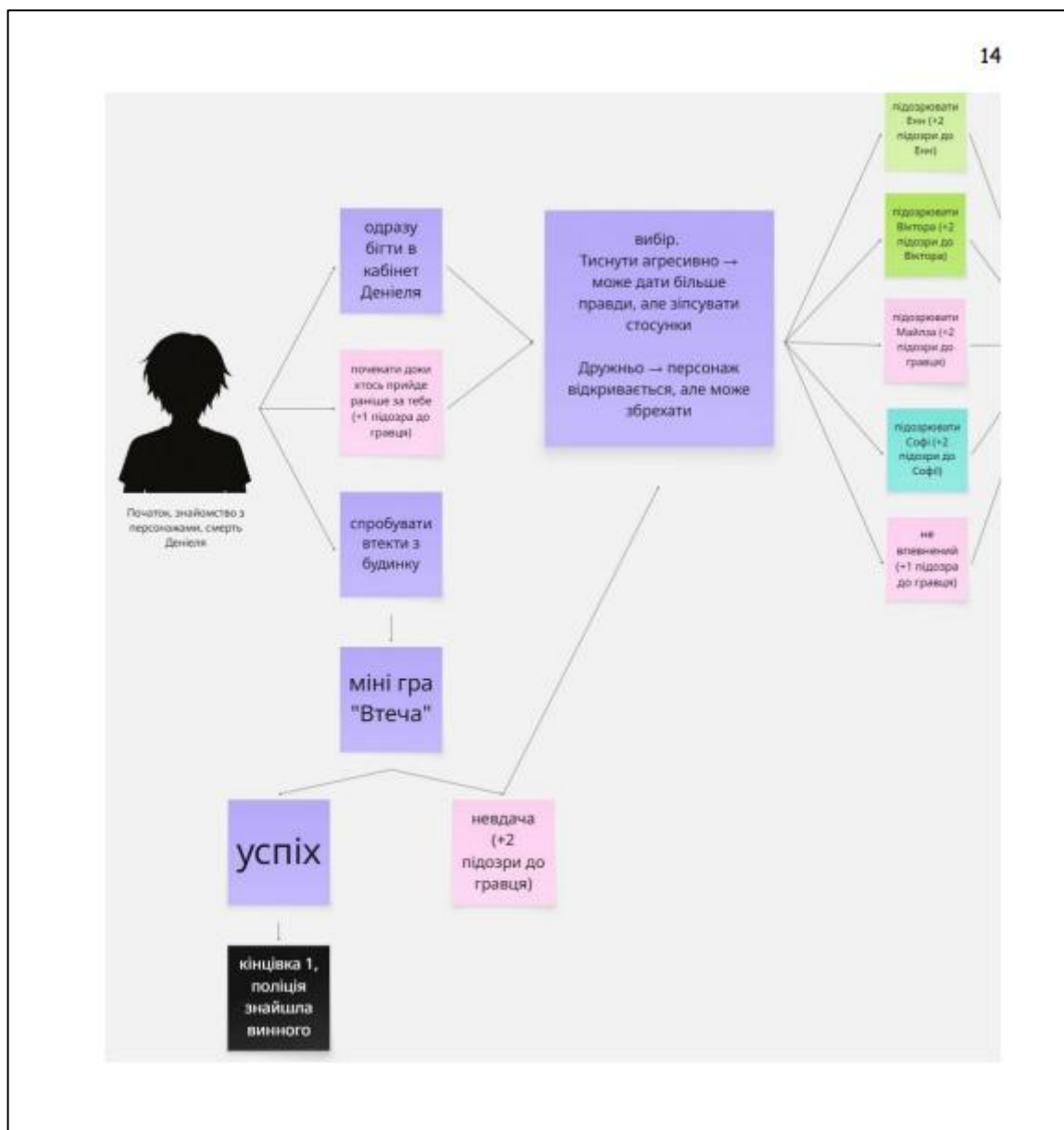
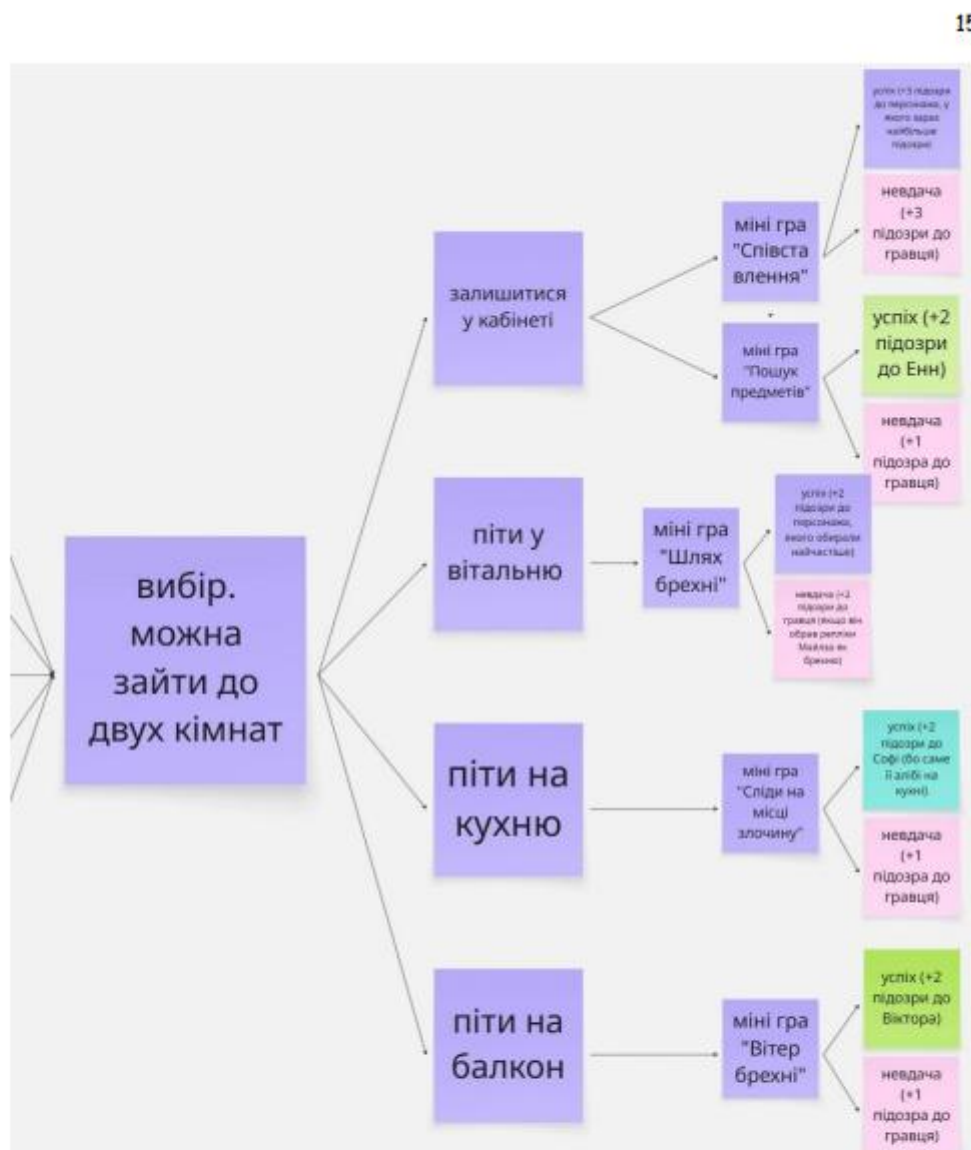


Рисунок В.14 – Сторінка 14 ГДД розроблюваної гри (скріншот виконаний самостійно)



8.5 Аудіо

Аудіо в «Midnight Deception» відіграє ключову роль у створенні атмосфери напруженого детективного трилера з елементами нуару та аніме стилю:

Рисунок В.15 – Сторінка 15 ГДД розроблюваної гри (скріншот виконаний самостійно)

- Фонова музика: для сцен гри обрано відповідні композиції, що підсилюють емоційний настрій та атмосферу.
- Звукові ефекти: деталі, такі як скрип дверей, кроки по підлозі, дзвін келихів, шурхіт паперів.

Приклади музики, яку можна використати:

- <https://www.fesliyanstudios.com/royalty-free-music/downloads-c/suspenseful-music/25>
- <https://pixabay.com/music/search/detective/>
- <https://wowsound.com/curated-playlists/visual-novel-slice-of-life-calm-relaxing-music.aspx>

Рисунок В.16— Сторінка 16 ГДД розроблюваної гри (скриншот виконаний самостійно)

ДОДАТОК Г

Рисунки до архітектури та проектування програмного забезпечення

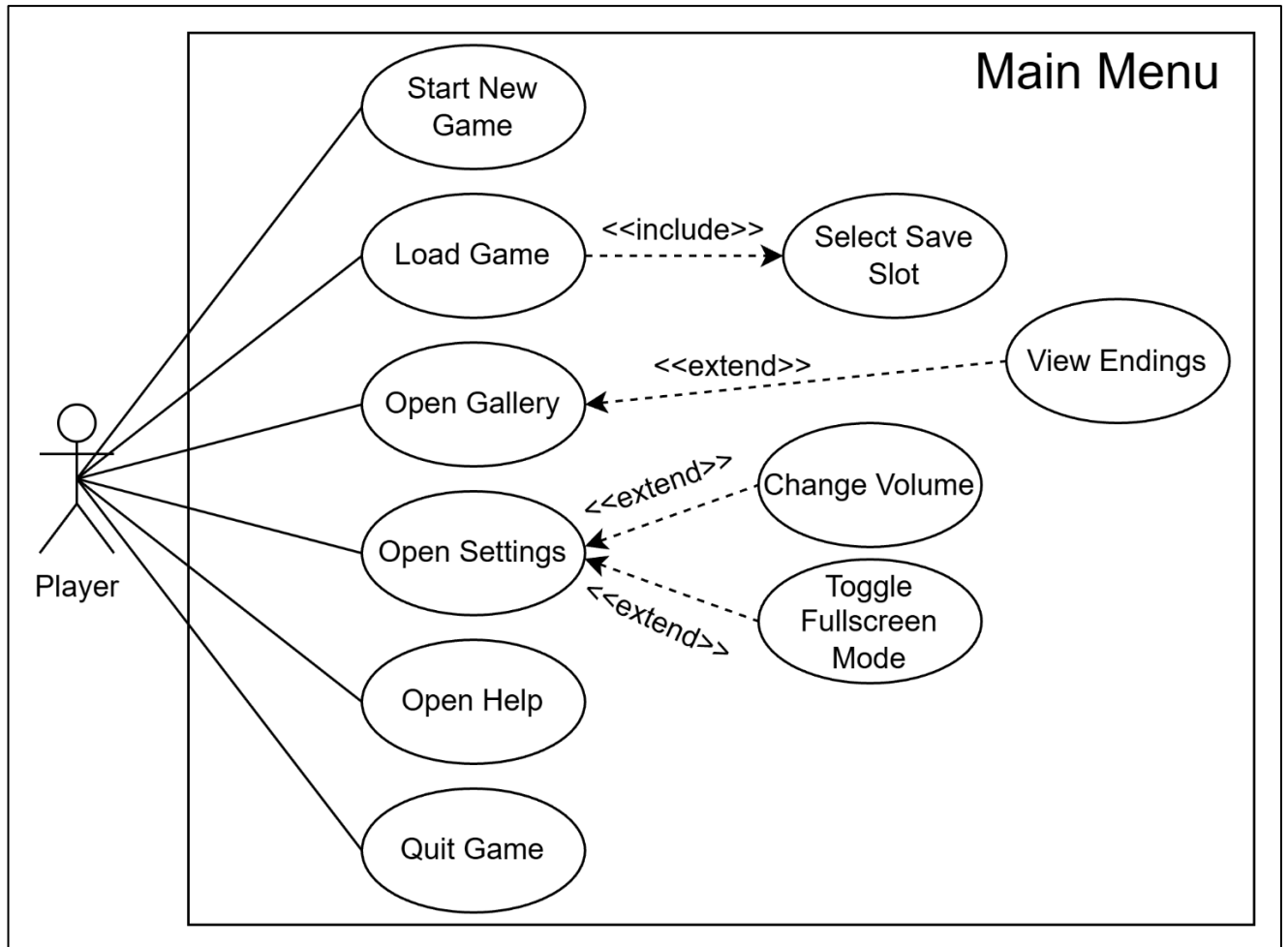


Рисунок Г.1 – Use Case Diagram головного меню гри (рисунок виконаний самостійно)

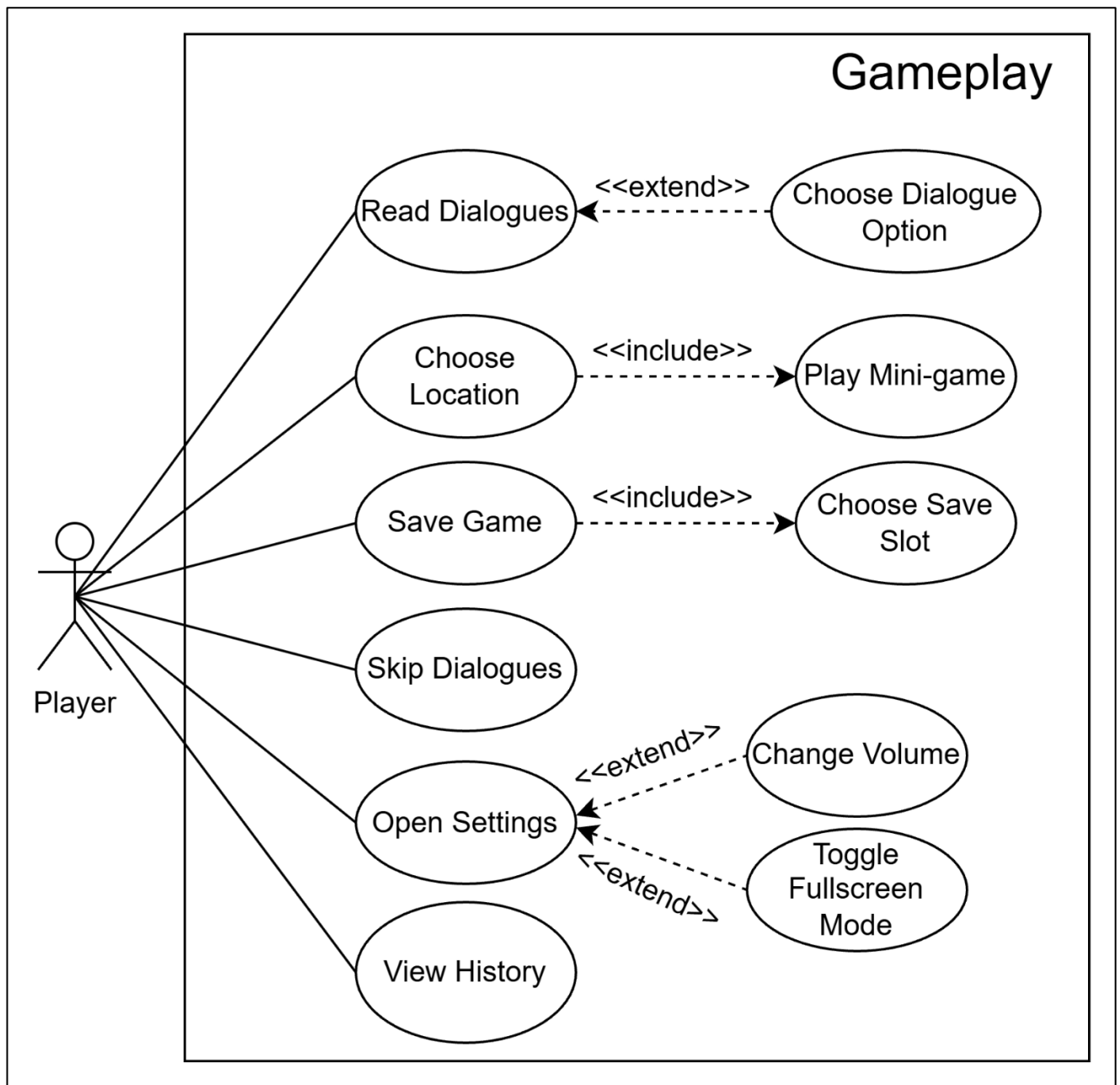


Рисунок Г.2 – Use Case Diagram ігрового процесу (рисунок виконаний самостійно)

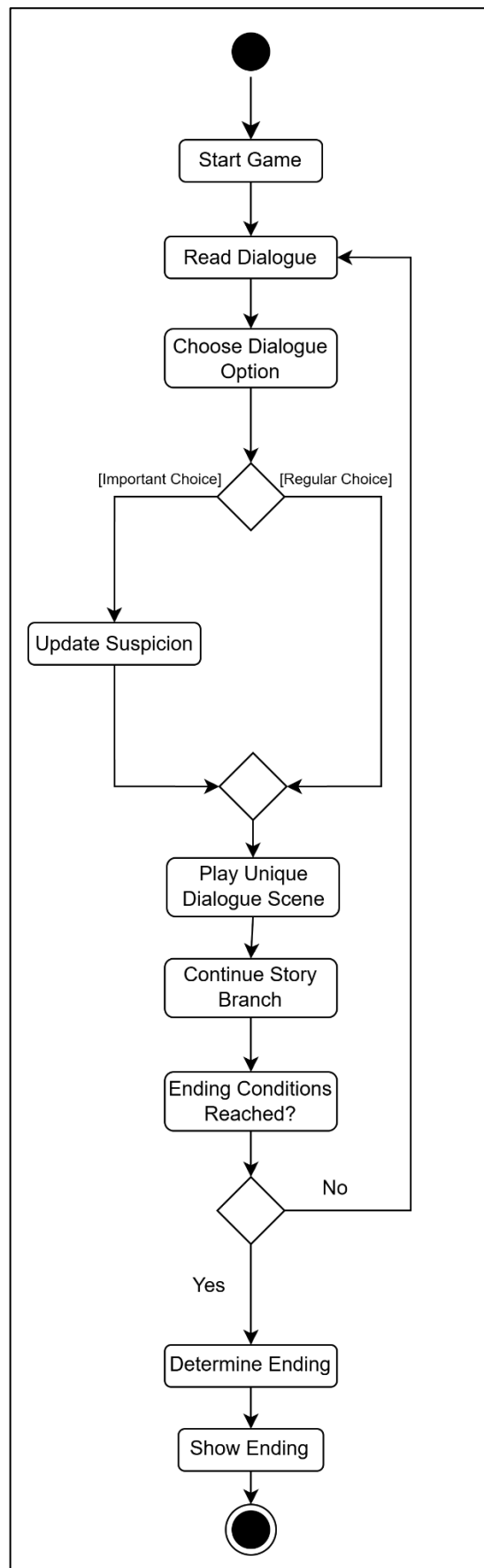


Рисунок Г.3 – Activity Diagram діалогів та розвитку сюжету (рисунок виконаний самостійно)

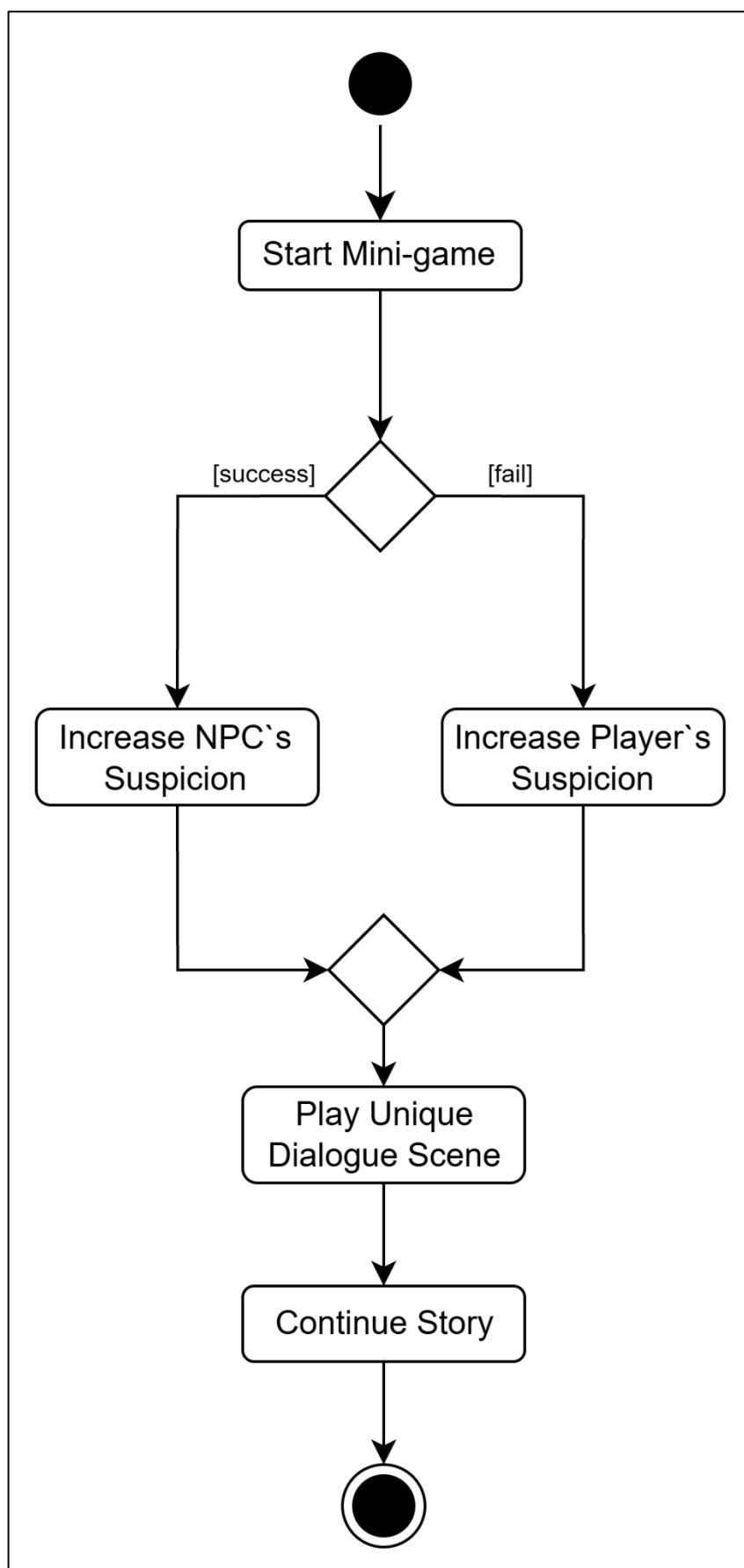


Рисунок Г.4 – Activity Diagram проходження міні-ігор (рисунок виконаний самостійно)

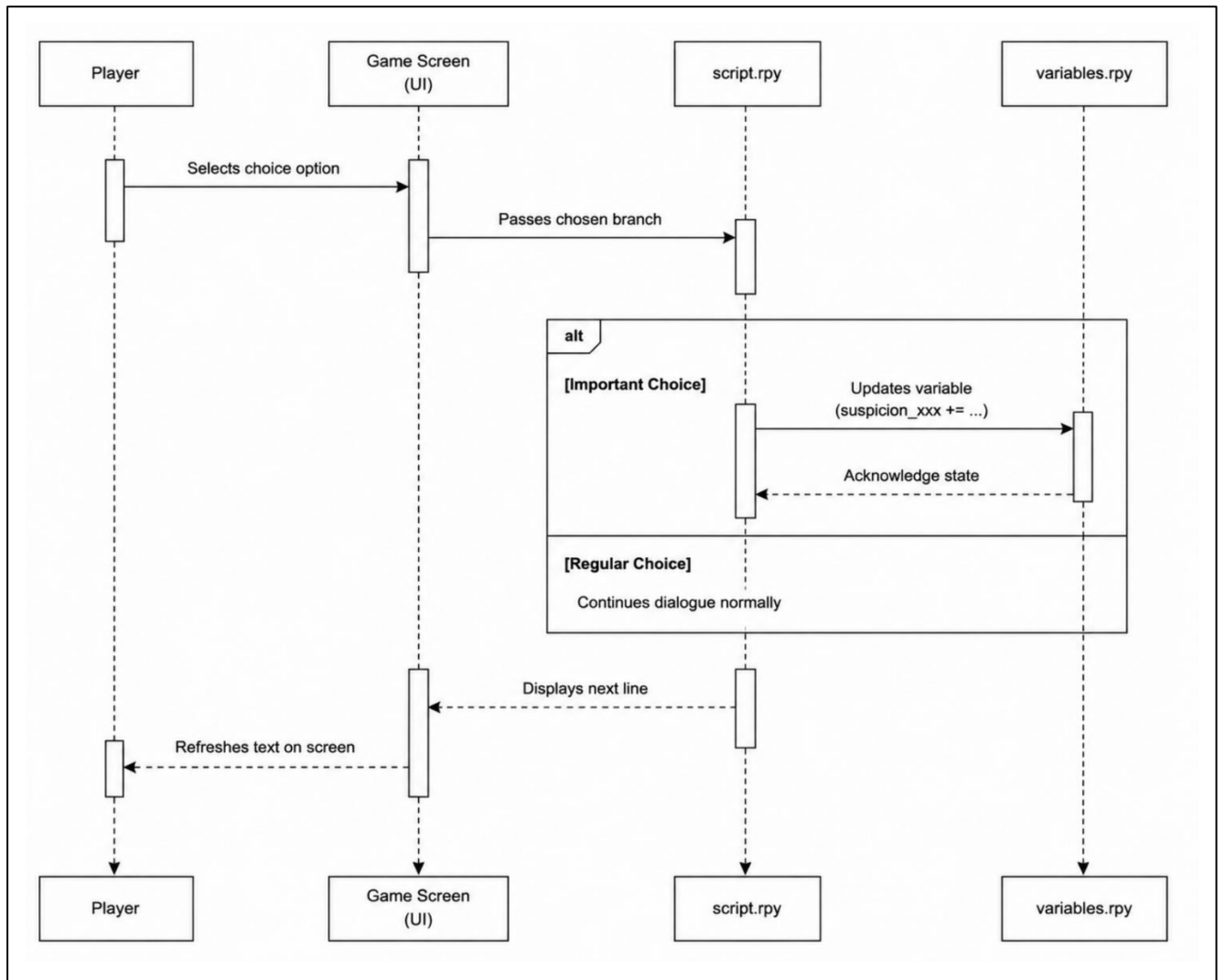


Рисунок Г.5 – Sequence Diagram вибору діалогу та зміни підозри (рисунок виконаний самостійно)

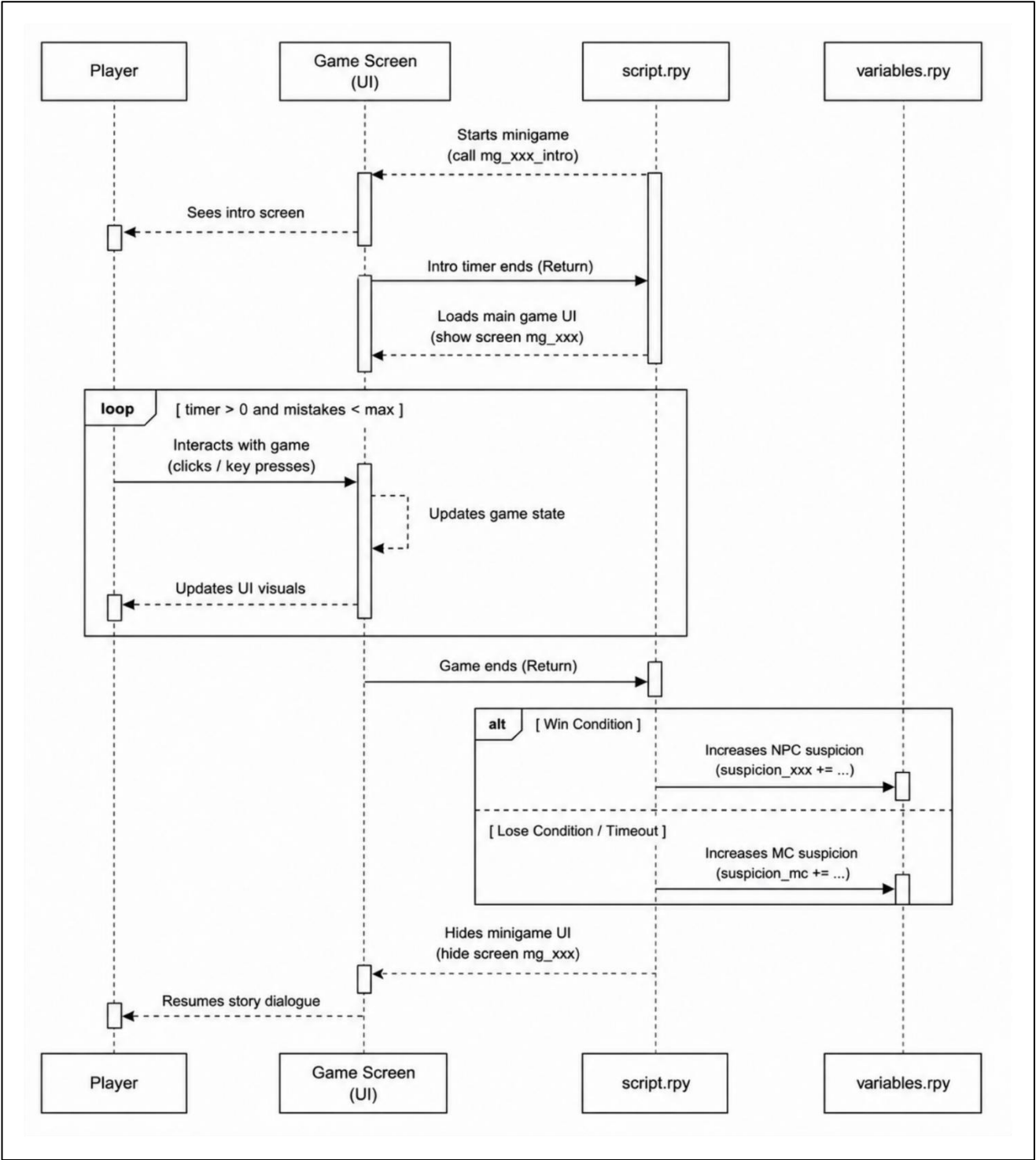


Рисунок Г.6 – Sequence Diagram запуску міні-гри та обробки результату (рисунок виконаний самостійно)

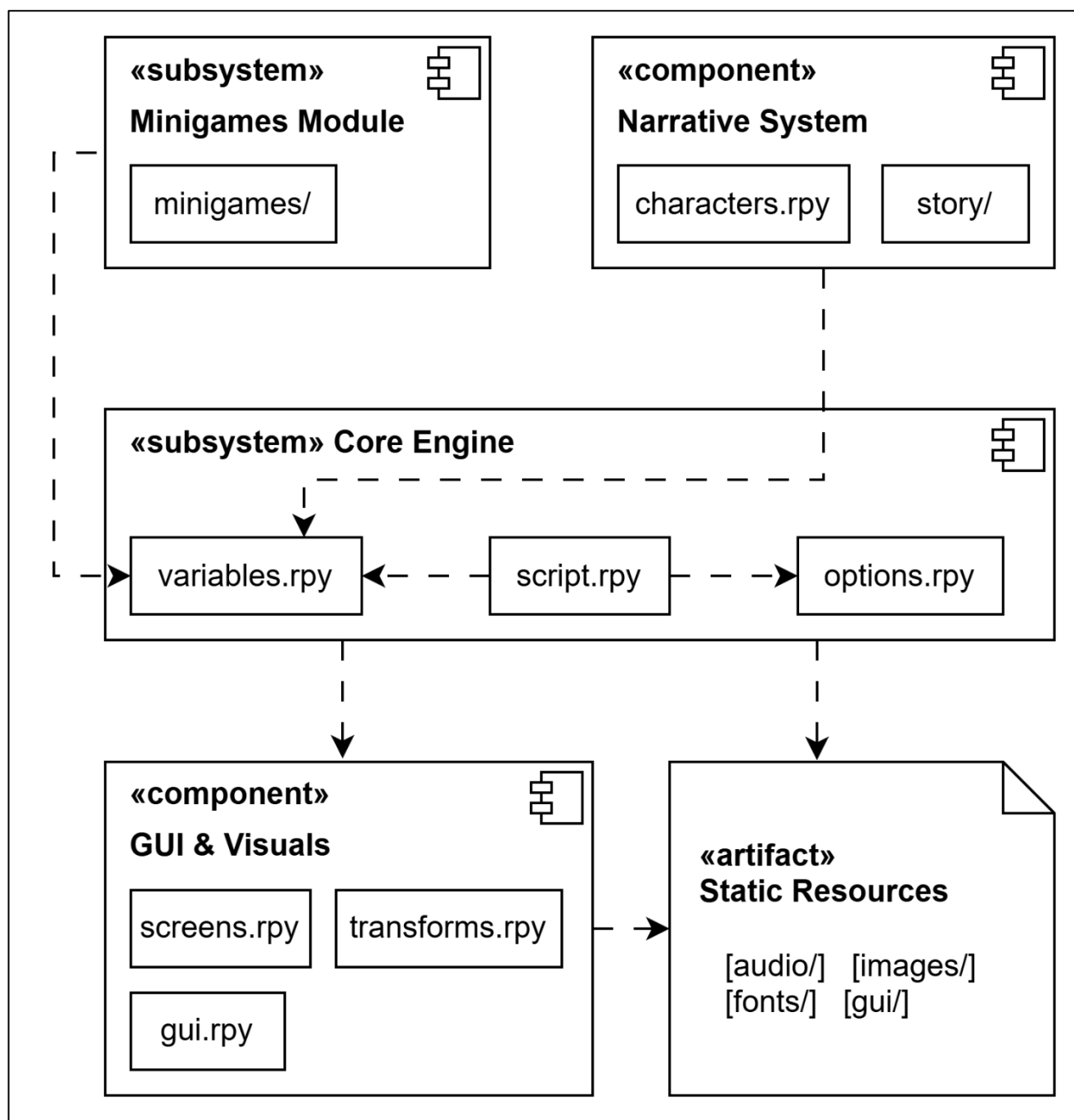


Рисунок Г.7 – Component Diagram архітектури гри (рисунок виконаний самостійно)

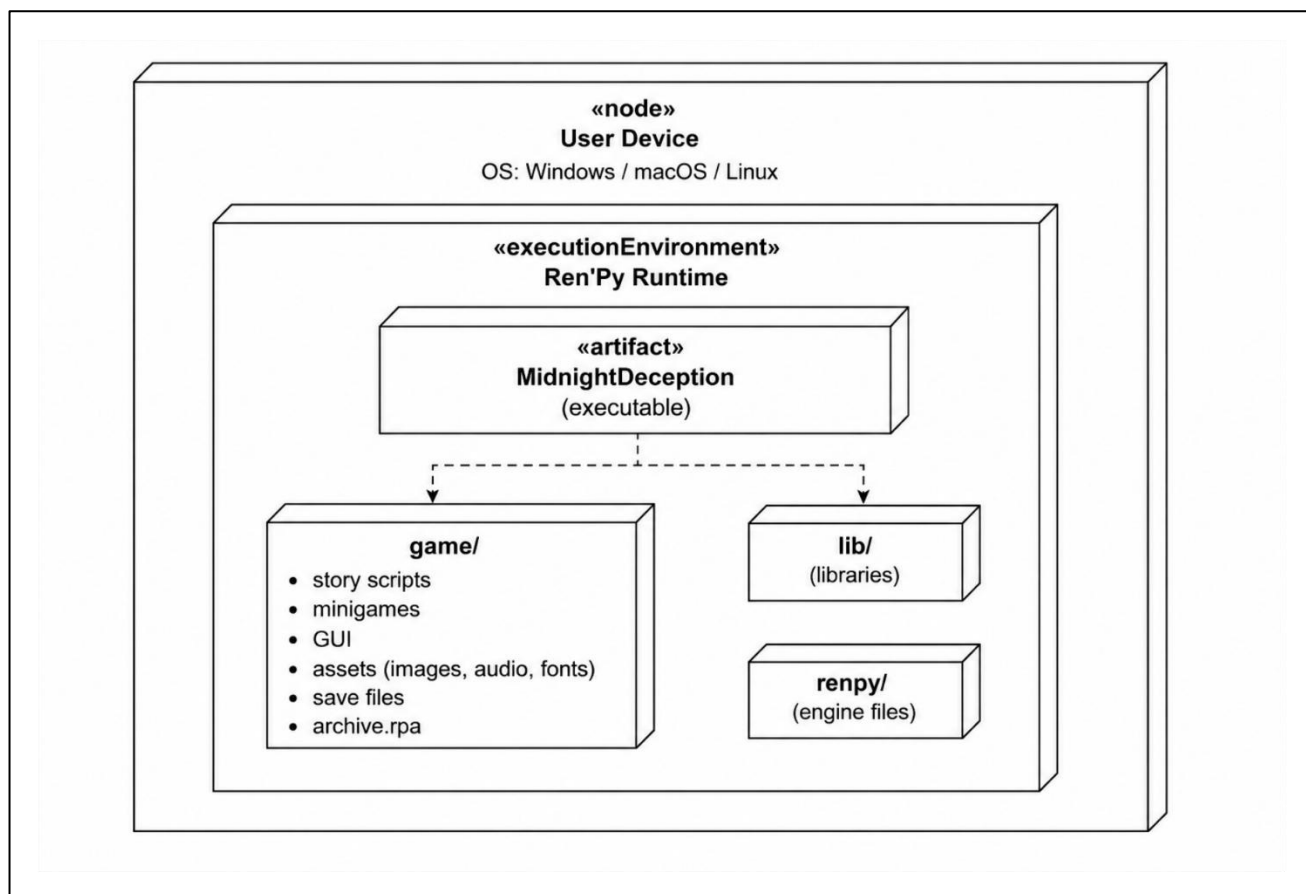


Рисунок Г.8 – Deployment Diagram архітектури гри (рисунок виконаний самостійно)